Toward Secure AI-Powered Penetration Testing Agents: Security Threats, Guardrails, and Architectural Perspectives

RAHUL DEV T Y, National Institute of Technology Calicut, India
HIRAN V. NATH✉, National Institute of Technology Calicut, India

arXiv:2609.16694v1 [cs.CR] 15 Sep 2026

LLM-powered autonomous agents are transforming the penetration testing space with dynamic, multi-step offensive security workflows that require minimal supervision by humans. These agents leverage sophisticated reasoning abilities and external security tools to independently carry out reconnaissance, identify vulnerabilities, devise exploitation plans, and perform post-exploitation operations. But the ability to have persistent memory, to take actions in the real world, and to do long-horizon reasoning raises qualitatively different security concerns than traditional chat-based LLM systems. Existing guardrail mechanisms for conversational AI may not be sufficient to secure autonomous AI pentesting agents accordingly.

To address these issues, we carry out a comprehensive security analysis on autonomous AI-penetration testing agents. We systematically analyse representative agent architectures, characterise their trust boundaries and attack surfaces and propose a threat taxonomy that is aligned with the lifecycle and covers LLM lifecycle attacks, agent-architecture attacks and cross-cutting behavioural attacks. We analyse the limitations of existing guardrail mechanisms, identify key research gaps, and discuss future research directions for developing specialised, context-aware, and architecture-aware guardrails to secure next-generation AI-driven offensive security systems.

CCS Concepts: • **Security and privacy** → *Intrusion/anomaly detection and malware mitigation*; **Intrusion detection systems**;

Additional Key Words and Phrases: AI Security, Large Language Models, AI-Powered Penetration Testing, Autonomous Agents, Attack Taxonomy, Threat Modeling, AI Guardrails, Prompt Injection, Memory Poisoning, RAG Poisoning, Supply-Chain Attacks, Multi-Agent Security, Training Data Poisoning, Agent Security

1 Introduction

The rapid growth of large scale software systems, cloud infrastructure and interconnected digital services has greatly increased the attack surface of modern computing environments. Penetration testing is a systematic, authorised, offensive security test that is critical to identifying exploitable vulnerabilities before an adversary does. But the scale, dynamism and technical breadth of modern systems has outstripped the ability of purely human-driven security assessments, creating a scalability problem and a persistent global shortage of skilled security practitioners. [90]

Recent advances in large language models (LLMs) have opened a new frontier in offensive security automation. LLMs exhibit strong multi-step reasoning, self-correction ability, and the ability to produce structured attack plans from high-level goals. When combined with agent frameworks that allow access to real-world tools [21, 123] such as network scanners, exploit frameworks, shell interpreters, and web browsers, LLMs can act as autonomous cyber operators that monitor the state of target systems, plan multi-stage attack sequences, execute tools, interpret results, and adapt strategies based on environmental feedback.

This has led to the rise of a new class of systems: *autonomous AI pentesting agents*. The state of this paradigm is exemplified by platforms like PentestGPT [18], AutoPentest [36], VulnBot [54], PenHeal [40], ARACNE [70], AutoAttacker [109], BreachSeek [2], Cochise [33], PentestAgent [79], HackSynth [69], AutoPentester [27], HackingBuddyGPT [31], and Incalmo [81] which demonstrate

Authors' Contact Information: Rahul Dev T Y, National Institute of Technology Calicut, Department of Computer Science and Engineering, Kerala, India, rahul_m251216cs@nitc.ac.in; Hiran V. Nath, National Institute of Technology Calicut, Department of Computer Science and Engineering, Kerala, India, hiranvnath@nitc.ac.in.

that artificial intelligence can automate tasks ranging from reconnaissance and vulnerability discovery to exploit generation and post-exploitation operations.

The benefits of these systems are impressive. But they introduce a new, under-appreciated class of security hazard: the attack surface is the AI system itself. A pentesting agent that can be gamed – in its inputs, its memory, its tool interfaces, or its underlying model training – is not just an ineffective tool, but an actively dangerous one that can execute real world cyber actions against unintended targets under adversarial control.

A major and largely unrecognised challenge is that **existing LLM guardrail mechanisms were designed for conversational artificial intelligence (AI), not for autonomous offensive-security agents that can execute real-world cyber actions.** Today's guardrails are based on the assumption of human supervision, single-turn interactions, trusted input channels, and text-only output. These assumptions are systematically violated by autonomous pentesting agents, which operate autonomously in multi-step campaigns, take input from adversarially-controlled target systems, and directly translate LLM output into shell commands and exploit tool invocations.

In this survey, we address this mismatch by proposing a **two-axis lifecycle-aligned taxonomy** based on the structure of the LLM security research literature. The taxonomy is arranged in two perpendicular axes. The first axis—LLM Lifecycle Attacks (Cat. A–C)—includes vulnerabilities inherited from the base model pipeline: pre-training and corpus poisoning (Cat. A), fine-tuning and alignment attacks (Cat. B), and inference-time prompt attacks (Cat. C). These vulnerabilities exist in any system that utilises an LLM, regardless of agent topology. The second axis—*Agent Architecture Attacks* (Cat. D–F)—concerns vulnerabilities that are not inherited but *emergent*, i.e., they only emerge when the agentic system adds persistent memory or RAG (Cat. D), inter-agent communication (Cat. E), or external tool execution (Cat. F). This is reflected in a symmetric six-category guardrail taxonomy (G-A to G-F) For each guardrail category, this survey systematically characterises not only whether existing mechanisms provide coverage, but *why* they are structurally insufficient in the pentesting agent context—a distinction not previously formalised in the literature.

1.1 Contributions

In brief, we present here the main contributions of this survey:

- We propose a unified analytical framework for security analysis of autonomous AI pentesting agents by systematically correlating architectural elements, trust boundaries, attack surfaces, failure modes, guardrail mechanisms and research challenges. The framework offers a systematic approach to examine how architectural design decisions impact the security stance of AI-based pentesting systems.
- We present a comprehensive architecture analysis of representative AI pentesting agents. We include and analyse key components such as reasoning and planning modules, memory management, retrieval-augmented generation (RAG), tool integration, reflection mechanisms, and multi-agent collaboration. This analysis allows us to identify architectural features that make these systems vulnerable to different security threats.
- We propose a taxonomy of security threats that includes the inherited vulnerabilities of foundation large language models (LLMs) and the emergent vulnerabilities of autonomous agent architectures. The proposed taxonomy provides a systematic organization of attacks across the LLM lifecycle and agent-specific operational layers, including memory and RAG poisoning, multi-agent coordination, and tool and workflow exploitation.
- We establish a systematic mapping from architectural components to attack surfaces to vulnerability categories to existing guardrail mechanisms. This analysis gives a detailed insight about the impact of architectural features on attack feasibility, the effectiveness

of current defence strategies and their limitations in securing autonomous AI pentesting agents.
- We analyse the limitations of existing guardrail approaches across different architectural layers and identify critical research gaps and directions for future research. Based on these observations, we discuss the key challenges and design principles for the development of robust, architecture-aware guardrails to secure next-generation autonomous AI pentesting systems.

The remainder of this paper is structured as follows. Section 2 presents the background on large language models, LLM-based agents, and the penetration testing lifecycle to provide the technical background for the survey. Representative autonomous AI pentesting frameworks are discussed in Section 3 and categorised into single-agent and multi-agent architectures, with their underlying models, planning mechanisms, memory usage, and tool integration capability studied (Table 1). Section 4 describes the threat model by defining the adversary capability tiers, trust boundaries, and attack assumptions discussed in this survey. Section 5 introduces the proposed two-axis attack taxonomy, distinguishing vulnerabilities originating from the *LLM Lifecycle Layer* (Categories A–C) and the *Agent Architecture Layer* (Categories D–F), together with a taxonomy of representative attacks (Fig. 1). Section 6 evaluates representative AI pentesting frameworks using the proposed taxonomy and analyzes their architectural vulnerability exposure. Section 7 presents the proposed guardrail taxonomy and assesses existing defense mechanisms through a comparative guardrail insufficiency analysis. Section 8 discusses the major research gaps and future directions identified from the attack and guardrail analyses. Section 9 concludes the paper.

Unlike existing surveys that focus on classifying attacks or summarising representative tools, this survey applies a unified analytical framework to systematically connect architectural components, trust boundaries, attack surfaces, failure modes and guardrail mechanisms. This enables a principled comparison of autonomous AI pentesting agents, and of architectural vulnerabilities and corresponding defence strategies.

Other recent efforts have surveyed adjacent aspects of this problem: agent security from a layered attack-surface perspective [14], offensive AI more broadly within cybersecurity [28], and prompt-sanitization-to-red-teaming pipelines for LLM agents [23]. Complementary reviews have also examined reinforcement-learning-based automated penetration testing [63] and general LLM-agent benchmarking practices for offensive security [32]. This survey differs from these efforts by adopting an architecture-centric, two-axis lifecycle taxonomy rather than a single-layer or tool-centric classification.

2 Background: AI-Based Penetration Testing Agents

Traditional penetration testing follows a structured life cycle of reconnaissance, scanning, vulnerability identification, exploitation, post-exploitation and reporting. In the past, each step was a great deal of human expertise. Partially individual subtasks were automated by the tool-assisted automation (Nmap, Metasploit, Burp Suite), but the cognitive center of the operation was kept in the human analyst.

Individual security tools and traditional automated penetration testing frameworks attempted to reduce the need for human effort by automatically building attack graphs, correlating vulnerabilities, and planning exploits. Tools like Cauldron are representative of a new generation of tools that take output from vulnerability scanners, network topology and exploit knowledge bases to build likely attack paths. This enables a systematic approach to risk assessment and remediation prioritisation [48] However, these frameworks heavily rely on pre-specified vulnerability databases,

symbolic reasoning engines and expert-designed attack models, which limit their ability to understand complex observations, generalise to unseen environments or perform autonomous multi-stage decision making. Hence, attack-graph-based systems had greatly improved the automation level of penetration testing, but they were still rule-based and human-supervised, not reasoning-based and autonomous.

This picture is fundamentally changed by LLMs. They can reason about complex technical artefacts like CVE descriptions, service banners, system logs, source code and infer their security implications. By coupling an LLM with external tools through agent frameworks we can close the loop between observation and action and enable autonomous execution of multi-stage attack campaigns.

Most importantly, the threat landscape of autonomous pentesting agents has evolved in tandem with the agents themselves. The LLM security literature now discusses attacks in all stages of the model lifecycle: From data poisoning before training [9], to alignment subversion during fine-tuning [5, 91], to runtime injection and tool exploitation during operation [16, 103]. The survey maps this threat landscape to the specific architectural properties of autonomous pentesting agents.

3 Representative AI Pentesting Tool Architectures

Autonomous AI pentesting agents represent a significant advancement over traditional LLM-assisted penetration testing systems. The first frameworks were based on a single reasoning agent that sequentially interpreted observations, generated attack strategies and called external security tools. Recently, architectures have leveraged persistent memory, retrieval-augmented generation (RAG), reflection mechanisms, autonomous tool orchestration, and collaborative multi-agent reasoning to address long-horizon penetration testing tasks. These architecture advances push the horizon of automation and reasoning capabilities but also introduce new trust boundaries and attack surfaces beyond the inherited vulnerabilities of foundation LLMs.

Rather than merely talking about the implementation details of AI pentesting frameworks, this survey takes an architecture-centric perspective. The security properties of autonomous pentesting agents are largely dictated by a common set of architectural abilities: reasoning, planning, memory management, knowledge retrieval, tool use, reflection, multi-agent cooperation. These abilities define not only how an agent behaves, but also its possible attack surface and the security requirements that arise from it. Thus, a systematic analysis of their vulnerabilities and guardrail mechanisms requires a first step of understanding the architectural characteristics of representative frameworks.

Representative AI pentesting frameworks can be roughly divided into *single-agent* and *multi-agent* architectures. Single-agent systems use a single reasoning engine to implement the penetration testing workflow through an observe–plan–act loop. Multi-agent systems distribute the penetration testing tasks to specialised agents responsible for planning, reconnaissance, retrieval, exploitation, validation or reporting. Both paradigms have the same goal of autonomous penetration testing, but differ significantly in architectural complexity, trust boundaries, communication mechanisms and security implications.

3.1 Single-Agent Systems

Single-agent AI pentesting frameworks centralize reasoning, planning, and tool execution within a single LLM instance. During each reasoning iteration, the agent observes the current environment, generates an attack strategy, invokes external security tools, interprets the resulting observations, and updates its internal context before determining the subsequent action. This tightly coupled reasoning process provides a relatively simple execution model while avoiding the complexity associated with inter-agent coordination.

ARACNE [70] proposes a shell-centric autonomous penetration testing architecture where a single LLM directly communicates with a terminal environment, generating commands, interpreting execution results, and iteratively refining next actions through an observe–plan–act reasoning process. The current implementation is mainly focused on reasoning and tool execution in an interactive shell without explicit persistent memory and retrieval-augmented reasoning components. The related work by the same authors has also shown autonomous Linux privilege-escalation attacks with LLMs [34], further bolstering the shell-execution risk profile discussed above.

AutoAttacker [109] uses a planner–executor workflow where one LLM manages attack context through rolling conversation history, repeatedly generating penetration testing actions.

CHECKMATE [94] formulates penetration testing as a single planning agent that sequentially generates, validates, and refines attack strategies throughout the assessment lifecycle.

Cochise [33] provides a standardized evaluation framework that cleanly separates the LLM backend from the execution environment, enabling reproducible benchmarking of autonomous penetration testing agents.

PenHeal [40] is a two-stage single-agent pipeline: a Pentest Module (Planner–Executor with an Instructor for knowledge retrieval) discovers vulnerabilities, which a Remediation Module (Estimator, Advisor, Evaluator) ranks and remediates under a cost budget.

PentestGPT [18] introduces modular reasoning by separating global planning from command generation while retaining a single-agent architecture. Human supervision remains on the execution path to validate generated actions before deployment.

HackSynth [69] implements an architecture for fully autonomous single-agent penetration testing composed of a planner and a summariser. Our framework can interact with external shell environments in a fully autonomous fashion, and it preserves long-horizon reasoning by continuously summarising the execution history, rather than relying on persistent memory or retrieval-augmentation.

RefPentester [15] improves the conventional observe–plan–act cycle with a dedicated self-reflection mechanism. The agent can reflect on its past actions and refine its future reasoning based on the feedback of execution.

HackingBuddyGPT [31] uses a lightweight architecture of a single agent in which an LLM is tightly coupled with an environment for executing SSH commands. The framework does not require persistent memory and retrieval mechanisms, as it is designed for interactive command generation for offensive security tasks, leading to decreased architectural complexity.

Overall, single-agent architectures offer relatively straightforward execution pipelines with centralized decision making. However, their tight coupling between reasoning and tool execution increases exposure to inference-time attacks such as prompt injection, observation poisoning, planner manipulation, and unsafe tool execution.

3.2 Multi-Agent Systems

Multi-agent AI pentesting frameworks decompose penetration testing into multiple specialized agents that cooperate to accomplish complex offensive security tasks. Instead of relying on a single reasoning engine, these architectures distribute responsibilities across planning, reconnaissance, knowledge retrieval, exploitation, validation, and reporting agents coordinated through dedicated orchestration mechanisms. Such decomposition improves modularity, scalability, and specialization while simultaneously introducing additional trust boundaries arising from inter-agent communication and shared reasoning state.

AutoPentest [36] employs a Supervisor agent to coordinate multiple specialized workers responsible for reconnaissance, knowledge retrieval, and vulnerability analysis.

AutoPentester [27], not to be confused with the similarly-named AutoPentest above, likewise follows a Supervisor–Worker design, in which a Supervisor agent delegates penetration testing subtasks to specialized worker agents while a retrieval-augmented generation component supplies prior attack strategies to guide command generation. Despite this multi-agent structure, reported subtask completion remains limited by recurring strategy-identification failures.

BreachSeek [2] adopts a Supervisor–Evaluator architecture in which specialized reconnaissance agents collect evidence that is validated before exploitation decisions are made.

PentestAgent [79] decouples reconnaissance and knowledge retrieval, by providing dedicated Reconnaissance and Search agents, allowing parallel information gathering.

Incalmo [81] is an LLM-agnostic multi-agent architecture that separates planning, environment modelling, and attack graph generation into distinct services. The modular decomposition enables flexible integration of different foundation models and improves architectural isolation between reasoning and execution components.

PENTEST-AI [7] is one of the most complete multi-agent architectures that integrates Scan-and-Search, Exploit Validation, Saga Controller and Zookeeper agents to coordinate penetration testing activities across multiple attack phases.

PTFusion [95] focuses on centralised coordination, retrieval and reflection in a MasterAgent, while reconnaissance tasks are delegated to dedicated ReconAgents.

ReaperAI [89] is an autonomous multi-phase architecture that executes reconnaissance, exploitation, privilege escalation, and post-exploitation tasks sequentially. The framework is centred on autonomous offence with the least possible human intervention.

VulnBot [54] separates the strategic planning from the execution, with the Planner and Executor agents sharing a common task representation.

CAI [67] is a highly extensible multi-agent penetration testing framework that combines a wide variety of foundation models and Model Context Protocol (MCP). It consists of several specialised agents, external plugins and heterogenous toolchains that offer flexible coordination across multiple offensive security workflows.

xOffense [65] uses a Task Orchestrator for campaign-level reasoning and Action Executors for specific penetration testing tasks, supported by an offense-oriented knowledge base.

Multi-agent systems provide a much better task decomposition, long-horizon reasoning and flexibility of operation than single-agent systems. However, these advantages are paid for in terms of increased architectural complexity and new attack surfaces through inter-agent communication, shared memory, distributed planning and collaborative decision making. Therefore, modern multi-agent pentesting systems need more robust guardrail mechanisms to secure the communication channels, preserve the reasoning integrity, and prevent cross-agent attack propagation.

3.3 Architectural Capability Comparison

The typical AI pentesting frameworks consist of a common set of architectural capabilities, but the details of implementation can be very different. The capabilities influence the autonomous agent's perception on the environment, reasoning on observations, retrieval of external knowledge, execution of security tools and orchestration of penetration testing activities. More importantly, they define the architectural trust boundaries through which adversarial inputs may pass and therefore define the attack surface of the system.

For systematic comparison, the security-relevant architectural capabilities of the surveyed AI pentesting frameworks are summarised in Table 1. This analysis focuses on architectural properties that directly impact security, namely planning and reasoning, persistent memory, retrieval augmented generation (RAG), reflection mechanisms, autonomous tool execution, multi-agent

Table 1. Architectural Capability Comparison of Representative AI Pentesting Frameworks

Framework	Planner	Memory	RAG	Reflection	Tool Execution	Multi-Agent	Human Oversight
ARACNE	✓	–	–	✓	✓	–	–
AutoAttacker	✓	Partial	✓	✓	✓	–	–
CHECKMATE	✓	–	–	✓	✓	–	✓
Cochise	✓	–	–	✓	✓	–	✓
HackSynth	✓	–	–	✓	✓	–	–
HackingBuddyGPT	✓	–	–	–	✓	–	–
PenHeal	✓	Partial	✓	✓	✓	–	✓
PentestGPT	✓	–	✓	✓	✓	–	✓
RefPentester	✓	Partial	–	✓	✓	–	✓
AutoPentest	✓	✓	✓	✓	✓	✓	Partial
BreachSeek	✓	✓	✓	✓	✓	✓	Partial
CAI	✓	✓	✓	✓	✓	✓	–
Incalmo	✓	✓	Partial	✓	✓	✓	Partial
PENTEST-AI	✓	✓	✓	✓	✓	✓	Partial
AutoPentester	✓	–	✓	✓	✓	✓	Partial
PentestAgent	✓	✓	✓	✓	✓	✓	Partial
PTFusion	✓	✓	✓	✓	✓	✓	Partial
ReaperAI	✓	Partial	–	–	✓	✓	–
VulnBot	✓	✓	✓	✓	✓	✓	–
xOffense	✓	✓	✓	✓	✓	✓	Partial

collaboration and oversight by humans, rather than implementation specific details. These architectural capabilities form the basis for the threat model and attack taxonomy presented in the subsequent sections.

The architectures surveyed reveal a clear evolution from centralised single-agent reasoning systems to collaborative multi-agent architectures with persistent memory, retrieval mechanisms, and autonomous tool orchestration. These architectural improvements significantly increase reasoning capabilities and penetration testing efficiency, but also introduce additional trust boundaries that increase the attack surface of autonomous AI pentesting agents. In particular, the opportunities of memory and retrieval poisoning are enabled by persistent memory and retrieval elements; workflow manipulation and unsafe command execution are raised by autonomous tool execution; and new attack vectors are introduced through inter-agent communication and distributed decision making by multi-agent collaboration. Thus, the architectural capabilities in Table 1 are used as a basis to derive the threat model and security taxonomy outlined in the next sections.

Beyond architectural capability, recent empirical work has begun questioning how effective these agents are in practice: [17] identifies the factors that distinguish capable LLM pentesting agents in real-world assessments, while [72] presents a comprehensive analysis suggesting that reported successes may partly reflect hallucinated findings rather than genuine exploitation.

4 Threat Model

In this survey, we consider a threat model where the *AI pentesting agent* is the object of analysis, as opposed to the target infrastructure being evaluated. This survey is concerned with the assessment of the autonomous agent itself, as opposed to traditional penetration testing which is aimed at identifying vulnerabilities in external systems. Hence, the attacker will try to influence the agent's reasoning, impair its decision making, pollute its knowledge sources, control its tool use or direct its offensive capabilities to unexpected targets.

This threat model builds upon the architectural capability analysis of the previous section. It looks at attacks in terms of the architectural trust boundaries they attack. These trust boundaries correspond to different stages of the AI pentesting pipeline and together define the attack surface of autonomous AI pentesting agents.

4.1 Adversary Access Tiers

The adversary is classified into three levels of capability based on the level of control over the AI pentesting system.

- **Tier 1 — Model-Level Adversary (Categories A–B).** The attacker has write access to the model development pipeline, including the pre-training corpus, supervised fine-tuning datasets, preference alignment data, or parameter-efficient adaptation mechanisms (e.g., LoRA adapters). Such adversaries can poison training data, manipulate alignment objectives, or propagate malicious model updates. The primary actors in this tier are supply-chain attackers, malicious dataset contributors, model providers, or insiders in the model development lifecycle.
- **Tier 2 —Runtime Context Adversary (Categories C–D):** The adversary does not have access to the internal model parameters but is able to influence the runtime context as perceived by the agent. This includes crafted prompts, adversarial target system responses, modified service banners, corrupted documentation, poisoned retrieval corpus items, or corrupted memory contents. This threat model is the most realistic for deployed AI pentesting agents, as the target infrastructure itself is an untrusted information source.
- **Tier 3 — Agent Architecture Adversary (Categories E–F):** The attacker targets the architectural elements that coordinate the autonomous agents. Such adversaries can impact the behaviour of downstream agents by tampering with inter-agent communication, injecting malicious tool definitions, registering compromised MCP servers or plugins, forging tool outputs, or interfering with execution workflows. This level is not about the foundation model per se, but the operational architecture.

4.2 Architectural Trust Boundaries

Based on the architectural capability analysis, three primary trust boundaries govern the security of autonomous AI pentesting agents.

- **Model Boundary.** Separates the foundation LLM from the agent framework constructed on top of it. Training-data poisoning, alignment manipulation, and malicious model updates exploit this boundary during model development.
- **Context Boundary.** Separates trusted system instructions from externally supplied runtime information, including user prompts, target-system observations, retrieved documents, and persistent memory. Runtime prompt injection, observation poisoning, memory poisoning, and retrieval poisoning exploit this boundary.

- **Execution Boundary.** Separates language generation from real-world execution. Tool invocation, shell command generation, plugin execution, API calls, and inter-agent coordination all cross this boundary, making it susceptible to workflow manipulation, unsafe tool execution, malicious plugins, and inter-agent attacks.

These trust boundaries establish the relationship between the architectural components discussed in Section 3 and the attack taxonomy presented in the following section. Consequently, every attack category analyzed in this survey can be interpreted as exploiting one or more architectural trust boundaries within the AI pentesting pipeline.

4.3 Scope

The proposed threat model explicitly excludes vulnerabilities within the target infrastructure being assessed, as these constitute the intended output of the penetration testing process rather than vulnerabilities of the AI pentesting agent itself. Furthermore, this survey does not consider physical attacks, hardware-level side-channel attacks, or operating-system compromise outside the execution environment of the autonomous agent.

5 Attack Taxonomy

Building upon the architectural capability analysis and threat model presented in the previous sections, this survey organizes attacks against autonomous AI pentesting agents into a unified two-axis taxonomy. Traditional LLM security surveys mainly classify attacks according to the machine learning lifecycle, while autonomous AI pentesting agents, in addition to inheriting vulnerabilities from foundation LLMs, also introduce new attack surfaces through persistent memory, retrieval mechanisms, multi-agent collaboration and autonomous tool execution. Consequently, a taxonomy designed solely around the LLM lifecycle is insufficient to characterize the complete threat landscape of AI-driven penetration testing systems.

The proposed taxonomy addresses this issue by segregating attacks on two complementary dimensions. The first dimension captures *LLM Lifecycle Attacks*, which target the underlying foundation model at pre-training, fine-tuning and alignment, or inference. Any AI pentesting framework, no matter how complex in its architecture, inherits all the attacks. The second dimension relates to *Agent Architecture Attacks*, arising from the architectural components used by autonomous agent frameworks, including persistent memory, retrieval-augmented generation (RAG), inter-agent communication, and autonomous tool execution. These attack classes are unique to agentic artificial intelligence systems, but do not exist in conventional standalone LLMs.

Figure 1 illustrates the relation between the proposed taxonomy and the threat model. Attacks on *Model Boundary* are mapped to Categories A–C, and those on the *Context Boundary* and *Execution Boundary* introduced by agent-based architectures to Categories D–F. Together, these two dimensions comprehensively cover the security landscape of autonomous AI pentesting agents, and lay the analytical foundation for the guardrail taxonomy in Section 7.

The first branch describes the attacks against the foundation model itself, through three stages of the life cycle. The most serious and long-lasting attack vector is pre-training attacks, where the attack occurs before the model is ever trained and can survive all the way through the alignment procedures [44, 119]. Training and alignment attacks require less privileged access (e.g., attacking instruction-tuning datasets, preference annotations or RLHF feedback pipelines), but affect all future users of the compromised model [5, 25, 37, 88, 91, 92]. Deployment and inference attacks require input-channel access only and encompass direct and indirect prompt injection [71, 113], supply-chain skill poisoning [75], optimization-based jailbreaks [55, 124], and the qualitatively new threat of large reasoning models acting as autonomous jailbreak agents [30]. A key insight is that

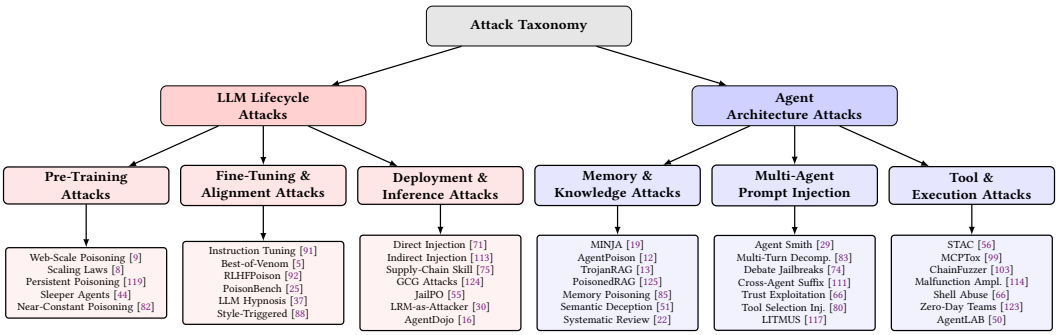


Fig. 1. Proposed two-axis taxonomy of attacks on autonomous AI pentesting agents. The first axis, *LLM Lifecycle Attacks* (Categories A–C), deals with vulnerabilities stemming from the foundation model across the model development lifecycle. The second axis, *Agent Architecture Attacks* (Categories D–F), captures new attack vectors enabled by persistent memory, retrieval mechanisms, multi-agent coordination, and autonomous tool execution. The exemplary attacks are shown for each category. Further discussion is given in the relevant subsections.

in pentesting scenarios the *policy proximity* problem—the infinitesimal semantic distance between an authorised and an unauthorised tool command—greatly undermines all inference-layer defences against this attack class.

The second branch captures attacks that emerge specifically from agentic architecture components. Memory and RAG poisoning attacks exploit the agent's unconditional trust in its own retrieved context: MINJA [19] achieves injection through query-only interaction, while AgentPoison [12] formalizes the attack as a constrained optimization over retrieval probability; TrojanRAG [13] and PoisonedRAG [125] further demonstrate that small numbers of adversarial passages are sufficient to hijack RAG outputs systematically. Multi-agent prompt injection attacks exploit inter-agent trust and shared communication channels: Agent Smith [29] demonstrates exponential worm-style propagation across agent ecosystems, while multi-turn decomposition [83], debate jailbreaks [74], and cross-agent suffix propagation [111] exploit coordination mechanisms that have no equivalent in single-agent systems. Tool and workflow exploitation attacks focus on the execution layer, where text turns into real-world consequence: STAC [56] demonstrates the chaining of benign tool calls to harmful sequences to bypass per-step guardrails; MCPTox [99] poisons MCP tool metadata to hijack agent planning; ChainFuzzer [103] discovers cross-tool data flows in which attacker-controlled content traverses multiple intermediate steps to get to high-impact operations.

5.1 LLM-Lifecycle Attacks

LLM lifecycle attacks target the foundation model throughout its development pipeline, including pre-training, post-training adaptation, and inference. These attacks are inherited by AI pentesting agents from the underlying LLM and remain applicable irrespective of the agent architecture. Consequently, compromising the foundation model at any lifecycle stage can affect every downstream AI pentesting framework built upon it.

5.1.1 Pre-training Attacks. Pre-training attacks focus on the first phase of the LLM development life cycle, where the integrity of the large-scale corpus used for training the foundation model is attacked. Since all further adaptation stages build on the knowledge obtained during pre-training, poisoned data can embed long-lasting malicious behaviours that cascade into downstream AI applications such as autonomous pentesting agents.

Target Component. The main attack surface is the pre-training corpus and the model optimisation process, where adversaries can inject large-scale datasets to influence the learned representations of the model.

Trust Boundary. These attacks violate the *Model Boundary*, as they compromise the foundation model before deployment, impacting all downstream systems built on top of it.

Representative Attacks. Representative techniques include data contamination, corpus poisoning and sleeper backdoor insertion. Recent work shows that poisoned training data can embed covert behaviours that survive fine-tuning and alignment [9, 44, 91], and pose a significant supply-chain risk to LLM-based systems.

Security Implications. Compromised pretraining of AI pentesting agents can lead to consequences such as persistent reasoning errors, malicious latent behaviours, and unsafe security recommendations that cannot be fully mitigated by inference-time guardrails alone.

5.1.2 Fine-Tuning and Alignment Attacks. Fine-tuning and alignment attacks focus on the adaptation stage after training, where a pre-trained foundation model is specialised through supervised fine-tuning (SFT), reinforcement learning from human feedback (RLHF), direct preference optimisation (DPO), or parameter-efficient tuning methods such as LoRA . Unlike pre-training attacks, these attacks change the behaviour of an existing model rather than its underlying knowledge.

Target Component. The main attack surfaces are finetuning datasets, preference annotations, reward models and parameter-efficient adaptation modules used to specialise the foundation model.

Trust Boundary. These attacks leverage the *Model Boundary* to compromise the adaptation pipeline between pre-training and deployment.

Representative Attacks. Representative techniques include instruction tuning poisoning, preference poisoning, reward model manipulation [102] and malicious LoRA adapters. Recent work shows that poisoning a small fraction of the fine-tuning data or alignment preferences can significantly change the model behaviour while remaining difficult to detect [5, 26, 91].

Security Implications. Compromising AI pentesting agents in fine-tuning or alignment can undermine safety measures, tilt vulnerability assessments, and lead to unsafe planning and tool use increasing the risk of erroneous or malicious penetration testing behaviour.

5.1.3 Inference-Time Attacks. Inference-time attacks on AI pentesting agents are performed during deployment by modifying the runtime context provided to the model. Unlike attacks in the training stage, these attacks do not change the model parameters, but leverage the framework's dependence on untrusted inputs. This makes them the most practical and widely observed threat to deployed LLM-based systems.

Target Component. The main attack surface includes the runtime context such as user prompts, target system responses, terminal outputs, service banners, retrieved documents, and other external information used in the reasoning process.

Trust Boundary. These attacks violate the *Context Boundary* by injecting malicious or misleading information into a model's reasoning context at the time of inference.

Representative Attacks. Representative techniques include prompt injection, jailbreak attack, prompt leakage, goal hijacking, context manipulation, and observation poisoning. Recent work shows that carefully-designed prompts or adversarial inputs can override system instructions, manipulate reasoning, or cause unsafe tool usage, all without modifying the underlying model [16, 73, 103].

Security Implications. Inference-time attacks for AI pentesting agents may divert the goals of penetration testing, generate unsafe commands, leak sensitive information, or produce deceptive security evaluations, thus compromising the trustworthiness and safety of autonomous penetration testing.

5.2 Agent Architecture Attacks

Autonomous AI pentesting agents have new attack surfaces beyond the vulnerabilities inherited from the foundation model: persistent memory, retrieval mechanisms, multi-agent collaboration, and autonomous tool execution. The addition of these architectural components introduces new trust boundaries that are not present in conventional standalone LLMs, resulting in a special class of agent-specific attacks.

5.2.1 Memory and Knowledge Attacks. Memory and knowledge attacks make use of external knowledge resources to improve the reasoning abilities of autonomous AI pentesting agents. These resources include persistent memory, retrieval-augmented generation (RAG), vector databases, and other long-term knowledge repositories. Unlike inference-time attacks, these attacks are across multiple reasoning iterations. Adversaries can therefore manipulate agent behaviour all the time.

Target Component. Primary attack surfaces include persistent memory stores, retrieval databases, vector indexes, and external knowledge repositories used to augment LLM reasoning.

Trust Boundary. These attacks violate the *Context Boundary* by injecting false information into the external memory before the information is fed into the reasoning process of the agent.

Representative Attacks. Some representative techniques are memory poisoning, retrieval poisoning, knowledge base poisoning, and vector database poisoning. Recent works have demonstrated that poisoned memory entries or retrieved documents can bias reasoning, propagate false knowledge, and impact subsequent planning and tool execution across multiple interactions [12, 13, 19].

Security Implications. Compromising the memory or retrieval system may cause AI pentesting agents to misidentify vulnerabilities, persistently reason incorrectly, repeatedly perform unsafe actions, and propagate malicious knowledge across the penetration testing workflow.

5.2.2 Multi-Agent Coordination Attacks. Multi-agent coordination attacks take advantage of the communication and cooperation mechanism among autonomous agents. Multi-agent architectures, unlike single-agent systems, communicate intermediate reasoning, observations and task assignment through inter-agent communication channels, thereby creating new attack surfaces which can impact the collective decision-making process.

Target Component. The main attack surfaces are the communication between agents, shared reasoning contexts, mechanisms for task delegation, and workflows for joint planning.

Trust Boundary. The exchanged information is manipulated, the *Execution Boundary* is compromised and the adversarial inputs can propagate through the multi-agent system attacking the cooperation between agents.

Representative Attacks. These techniques include communication poisoning, inter-agent prompt injection, trust exploitation, debate manipulation, and cross-agent jailbreak propagation. Recent work has shown that malicious messages from a single compromised agent can affect downstream reasoning and propagate unsafe behaviours among collaborating agents [29, 56]

Security Aspects. The absence of coordination among AI pentesting agents may lead to wrong task assignment, dissemination of malicious logic, unsafe exploit choices, and synchronised execution of adversarial actions, thereby severely decreasing the reliability of collaborative penetration testing.

5.2.3 Tool and Execution Attacks. Tool and execution attacks exploit the interaction between autonomous AI pentesting agents and external tools, plugins, APIs and execution environments. Unlike classic attacks that target the reasoning of the model, these attacks target the execution pipeline, allowing an attacker to manipulate the tool selection, command execution, workflow orchestration or interaction with external services.

Target Component. The main attack vectors are tool interfaces, shell execution modules, plugin ecosystems, Model Context Protocol (MCP) servers, external APIs, and components for orchestrating workflows that execute actions created by agents.

Trust Boundary. Such attacks undermine the *Execution Boundary* by converting LLM-synthesized actions into real-world tools, leading to the impact of malicious inputs on operational workflows.

Representative Attacks. These include metadata poisoning of tools, malicious MCP servers, workflow manipulation, command injection via tool outputs, compromised plugins, and API exploitation. Representative techniques include recent work demonstrating that adversarial responses from tools, or a compromised execution environment, can distract agents to choose wrong tools, execute unsafe commands, or propagate malicious actions through complex penetration testing workflows [16, 103].

Security Implications. Successful tool and execution attacks against AI pentesting agents could result in unauthorised command execution, exploitation of wrong vulnerabilities, corrupted assessment results, and unintended interactions with external systems. As autonomous agents increasingly rely on external tools to perform security assessments, securing the execution pipeline is critical to safe and reliable operation.

5.3 Hybrid Attacks

Some papers fall into more than one taxonomy class and their hybrid nature has important implications for defence design. Agent Smith [29] merges the deployment-layer multimodal injection (Cat. C) and exponential multi-agent propagation (Cat. E). PoisonedSkills/DDIPE [75] combines skill poisoning of the supply chain (Cat. C) and exploitation of tools and workflows (Cat. F). STAC [56] combines tool and workflow exploitation (Cat. F) with long-horizon sequential planning. LLM Hypnosis [37] mixes gradual behavioural drift over sessions with alignment-layer preference corruption (Cat. B). The spread of hybrid attacks has a structural implication: any defence that is working on a single taxonomy layer is not enough. A memory-integrity guard that catches Cat.D injection is not a prevention for STAC (Cat.F); a run-time tool monitor is not a prevention for supply-chain skill poisoning (Cat.C). To defend effectively, a coordinated cross-layer coverage is required.

Automation-Exploit [3] is also a hybrid solution, combining multi-agent offensive planning (Cat. E) with digital-twin-based risk-mitigated exploitation, and showing how defensive sandboxing can be coupled with autonomous attack generation. Learning-based automated adversarial red-teaming [118] blurs the boundary between inference-time attack generation (Cat. C) and systematic robustness evaluation further.

6 Attack Vulnerability Assessment of AI Pentesting Tools

This section complements the proposed attack taxonomy with a qualitative architectural vulnerability assessment of representative AI-based pentesting frameworks. Rather than measuring empirical attack success rates, the assessment takes into account the potential exposure of each framework to the six attack categories (Categories A–F) based on the architectural characteristics identified in Section 3 and the threat model presented in Section 4. Therefore the evaluation should be viewed as a security analysis based on the architecture and not a live penetration testing benchmark.

The vulnerability matrices are arranged according to the two-axis taxonomy introduced in Section 5. The categories A–C correspond to *LLM Lifecycle Attacks* inherited from the underlying foundation model regardless of the agent architecture. Categories D–F, however, are *Agent Architecture Attacks*. These attacks only become apparent if persistent memory, retrieval, multi-agent coordination, and autonomous tool execution exist.

To make the discussion more clear, representative frameworks are divided into single-agent and multi-agent architectures. Table 2 shows the vulnerability assessment for the single agent systems. Table 3 summarises the vulnerability assessment for multi-agent and hybrid architectures. This separation is because some types of attacks, especially multi-agent coordination attacks (Category E), are not applicable in structure to frameworks that do not use multiple cooperative agents.
Assessment Methodology. Ratings for each vulnerability are derived from the architectural capabilities described in the respective framework publications, summarised in Table 1. The assessment takes into account three major factors:

- **Dependency on Foundation Model (Categories A–C):** Security properties of the underlying foundation model, including its training pipeline, post-training alignment and susceptibility to inference-time attacks.
- **Knowledge Management (Category D):** Persistent memory, retrieval-augmented generation (RAG), vector databases, or other external knowledge repositories that extend the agent's reasoning context.
- **Execution Architecture (Categories E–F):** The level of autonomous agent coordination, inter-agent communication, tool integration, and direct execution of LLM-generated actions.

Based on these architectural characteristics, each framework is assigned one of four qualitative ratings:

- **High (H):** Direct architectural exposure with no explicitly reported mitigation.
- **Medium (M):** Plausible exposure mitigated through partial architectural safeguards, constrained execution, or human oversight.
- **Low (L):** Limited exposure due to explicit architectural constraints or dedicated defensive mechanisms.
- **Not Applicable (–):** The corresponding attack category is structurally inapplicable because the required architectural component is absent (e.g., Category E in single-agent systems).

It is important to emphasize that Categories A–C primarily represent *inherited* vulnerabilities associated with the underlying foundation model throughout its lifecycle. Therefore, their exposure is mostly dictated by the security properties of the underlying model, i.e. the integrity of the pre-training pipeline, post-training alignment and robustness to inference-time attacks. Instead, *Emergent* vulnerabilities are represented by Categories D–F that are a consequence of the architectural design of autonomous AI pentesting agents including persistent memory, retrieval mechanisms, multi-agent coordination, and autonomous tool execution.

The resulting vulnerability matrices provide a systematic mapping between the proposed attack taxonomy and representative AI pentesting frameworks, identifying the architectural attack surfaces that motivate the guardrail taxonomy presented in the following section.

7 Guardrail Mechanisms for AI Pentesting Agents

The guardrail taxonomy is intentionally organized to mirror the attack taxonomy, enabling systematic mapping between attack surfaces and defense mechanisms rather than representing an independent classification.

While substantial progress has been made in developing guardrail mechanisms for large language models (LLMs), most existing approaches have been designed for conversational assistants, content moderation, or general-purpose AI applications. Autonomous AI pentesting agents operate in a fundamentally different environment, where they must reason over untrusted observations, maintain long-term memory, coordinate with other agents, and autonomously interact with external security tools. Consequently, guardrails designed for conventional LLMs are insufficient to address the broader attack surface introduced by agentic AI systems.

Table 2. Architecture-based qualitative vulnerability assessment of representative single-agent AI pentesting frameworks according to the proposed attack taxonomy. Attack categories: A=Pre-training Attacks, B=Fine-Tuning and Alignment Attacks, C=Inference-Time Attacks, D=Memory and Knowledge Attacks, E=Multi-Agent Coordination Attacks, F=Tool and Execution Attacks.

Framework	Representative Base Model	Architecture	Attack Category						Architectural Security Observations
			A	B	C	D	E	F	
PentestGPT [18]	GPT-4 / GPT-4o	Penetration Testing Tree (PTT) + Human in the Loop (HITL)	M	M	H	L	–	M	PTT vulnerable to context poisoning; human oversight reduces unsafe execution; no persistent memory; moderate tool exposure.
AutoAttacker [109]	GPT-4	ReAct + Retrieval-Augmented Generation (RAG)	M	M	H	H	–	H	RAG introduces memory poisoning risk; autonomous tool execution increases execution-layer exposure.
ARACNE [70]	GPT-4o	Planner + Interpreter	L	L	M	L	–	M	Prompt-injection defenses reduce inference risk; SSH execution remains an execution attack surface.
HackSynth [69]	GPT-4o / Llama-3	Planner + Command Generation + Summarizer	L	L	H	L	–	H	No persistent memory; autonomous shell execution dominates the attack surface.
PenHeal [40]	GPT-4	Pentest + Remediation	L	M	H	M	–	M	Counterfactual reasoning improves robustness, but generated remediation may still be influenced by adversarial observations.
HackingBuddyGPT [31]	GPT-4	Single-Agent Shell Executor + Feedback	L	L	H	L	–	H	Minimal architecture reduces memory attacks but direct shell interaction creates significant execution risk.
CHECKMATE [94]	GPT-4	Planner + Validator	L	L	M	L	–	M	Validation-guided planning improves reasoning robustness; autonomous tool execution remains the primary execution-layer attack surface.
Cochise [33]	LLM-agnostic	Planner + Tool Execution + Benchmark Evaluation	L	L	M	L	–	L	Controlled benchmark environment minimizes memory-related attacks while limiting exposure to unsafe autonomous execution.
RefPentester [15]	GPT-4	Planner + Reflection	L	M	M	L	–	M	Explicit self-reflection improves planning quality but does not eliminate prompt injection or unsafe tool invocation.

Note: Ratings indicate qualitative architectural exposure derived from the published framework designs and should not be interpreted as empirical attack success rates.

Legend: **H** High architectural exposure; **M** Moderate architectural exposure; **L** Low architectural exposure; – Attack category structurally not applicable.

To systematically organize existing defense mechanisms, this survey proposes a two-axis guardrail taxonomy that mirrors the attack taxonomy presented in Section 5. The first axis, *LLM Lifecycle Guardrails* (Categories G-A–G-C), protects the foundation model throughout its development lifecycle, including pre-training, fine-tuning and alignment, and inference. The second axis, *Agent Architecture Guardrails* (Categories G-D–G-F), protects the architectural components introduced by autonomous AI agents, including persistent memory, retrieval mechanisms, multi-agent coordination, and external tool execution.

Unlike conventional LLM guardrails that primarily focus on preventing unsafe text generation, the proposed taxonomy extends protection across multiple architectural trust boundaries. Accordingly, each guardrail category is designed to mitigate the primary attack surface identified in the corresponding attack taxonomy, thereby establishing a one-to-one mapping between attacks and defenses.

Figure 2 illustrates the proposed guardrail taxonomy. Categories G-A–G-C mitigate vulnerabilities inherited from the foundation model lifecycle, whereas Categories G-D–G-F protect the agent-specific architectural components that enable autonomous penetration testing. Together, these guardrail categories provide a comprehensive defense framework for securing AI-based pentesting agents against both inherited LLM vulnerabilities and emergent agent-specific attacks.

Table 3. Architecture-based qualitative vulnerability assessment of representative multi-agent AI pentesting frameworks according to the proposed attack taxonomy. Attack categories: A=Pre-training Attacks, B=Fine-Tuning and Alignment Attacks, C=Inference-Time Attacks, D=Memory and Knowledge Attacks, E=Multi-Agent Coordination Attacks, F=Tool and Execution Attacks.

Framework	Base LLM	Architecture	Attack Category						Architectural Security Observations
			A	B	C	D	E	F	
VulnBot [54]	GPT-4o / Llama-3	Penetration Testing Graph(PTG) + Retrieval-Augmented Generation(RAG) + Multi-Agent.	M	M	H	H	H	H	Persistent memory and PTG coordination increase exposure to memory, coordination, and execution attacks.
BreachSeek [2]	GPT-4o / Claude 3.5	Supervisor + Specialist Agents	M	M	H	M	H	H	Shared task context and orchestrator communication create propagation paths for coordination attacks.
PentestAgent [79]	GPT-4	Planning Agents + RAG + Execution History	M	M	H	H	H	H	Knowledge retrieval and inter-agent communication expose both memory and coordination attack surfaces.
Incalmo [81]	LLM-agnostic	Planner + Attack Graph + Specialist Agents.	L	L	M	M	M	M	Intent abstraction reduces direct prompt attacks, while shared attack graphs remain susceptible to manipulation.
ReaperAI [89]	GPT-4	Autonomous Multi-Phase.	M	M	H	M	H	H	Autonomous execution and phase coordination significantly increase execution-layer attack exposure.
CAI [67]	Multi-LLM	Multi-Agent + Model Context Protocol (MCP).	H	H	H	H	H	H	Supports numerous foundation models and external MCP tools, resulting in the broadest architectural attack surface among surveyed frameworks.
AutoPentest [36]	GPT-4 / GPT-4o	Penetration Testing Tree (PTT) + Autonomous Multi-Agent.	M	M	H	M	H	H	Autonomous planning and distributed execution increase exposure to coordination failures and unsafe tool invocation.
PENTEST-AI [7]	GPT-4	MITRE ATT&CK-Guided Multi-Agent.	M	M	H	H	H	H	Multiple collaborative agents guided by MITRE ATT&CK taxonomy expand the attack surface through shared reasoning, memory exchange, and autonomous execution.
AutoPentester [27]	GPT-4 / GPT-4o	Planner + Supervisor + Specialized Workers + Retrieval-Augmented Generation (RAG).	M	M	H	H	H	H	Supervisor-coordinated workers and RAG-based strategy retrieval increase exposure to memory, coordination, and execution attacks; low subtask completion reported due to strategy identification failures.
PTFusion [95]	GPT-4	MasterAgent + ReconAgents.	M	M	H	H	H	H	Distributed reconnaissance and shared memory improve coverage but expose the framework to coordination and memory poisoning attacks.
xOffense [65]	Multi-LLM	Collaborative Multi-Agent.	H	H	H	H	H	H	Heterogeneous multi-agent collaboration and extensive tool integration create a broad attack surface across all taxonomy categories.

Note: Ratings indicate qualitative architectural exposure derived from the published framework designs and should not be interpreted as empirical attack success rates.

Legend: H High architectural exposure; M Moderate architectural exposure; L Low architectural exposure; – Attack category structurally not applicable.

7.1 G-A: Pre-Training Guardrails

Pre-training guardrails protect the integrity of the foundation model by securing the large-scale corpus and optimization pipeline used during pre-training. Since vulnerabilities introduced at this stage propagate to all downstream applications, these guardrails aim to detect and eliminate poisoned data, hidden backdoors, and malicious training samples before the foundation model is deployed.

Protected Component. The primary protected components include the pre-training corpus, data collection pipeline, and model optimization process.

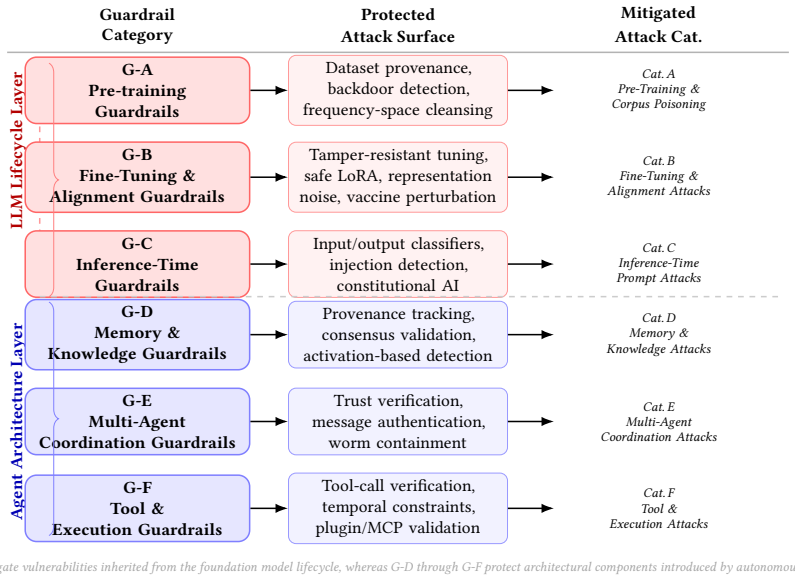


Fig. 2. Proposed guardrail taxonomy for autonomous AI pentesting agents, organized along the same two-axis structure as the attack taxonomy. Categories G-A–G-C mitigate vulnerabilities inherited from the LLM lifecycle, whereas Categories G-D–G-F protect the architectural components introduced by autonomous agent frameworks. Each guardrail category is designed to defend the primary attack surface associated with its corresponding attack category, establishing a one-to-one mapping between the proposed attack and defense taxonomies.

Protected Trust Boundary. G-A guardrails protect the *Model Boundary* by ensuring that the foundation model is trained using trustworthy data prior to deployment.

Representative Defense Mechanisms. Representative approaches include dataset provenance verification (Gracefully) [105] that identifies poisoned samples through importance-weighted resampling, activation-space backdoor detection [61] that detects hidden trigger behaviours by analysing internal model representations, and chain-of-thought based backdoor detection [59] that exploits reasoning discontinuities exhibited by compromised models. The combination of these techniques results in the improved integrity of the pre-training pipeline by identifying malicious training data before model optimisation. Another line of work focuses on improving clean-model recovery after poisoning by downscaling frequency-space artefacts in backdoored training data [106], which provides a corpus-cleansing mechanism different than provenance-based filtering.

7.2 G-B: Fine-Tuning and Alignment Guardrails

Fine-tuning and alignment guardrails protect the post-training adaptation pipeline by ensuring that supervised fine-tuning (SFT), reinforcement learning from human feedback (RLHF), direct preference optimization (DPO), and parameter-efficient fine-tuning methods preserve the safety and alignment properties of the foundation model. These guardrails seek to prevent adversarial manipulation of the model during specialization while maintaining its intended behavior.

Protected Component. The primary protected components include fine-tuning datasets, preference annotations, reward models, alignment objectives, and parameter-efficient adaptation modules such as LoRA adapters.

Protected Trust Boundary. G-B guardrails protect the *Model Boundary* by securing the post-training adaptation pipeline before the model is deployed.

Representative Defense Mechanisms. Representative approaches include tamper-resistant safety fine-tuning [86], which improves the robustness of aligned models against subsequent malicious fine-tuning, Safe LoRA [39], which constrains adapter updates within a safety-preserving subspace, PEFTGuard [84], which audits parameter-efficient adapters for malicious modifications, representation noising [77], which reduces the effectiveness of backdoor triggers through controlled perturbations of latent representations, Vaccine [43], which pre-immunizes models against alignment attacks using simulated adversarial exposure, and SEAL [78], which improves alignment robustness through careful selection of fine-tuning data. Further defenses in this space include Antidote [41] and Booster [42], which counteract harmful fine-tuning by attenuating malicious gradient perturbations post-hoc; SaLoRA [57], which preserves safety alignment within low-rank adaptation updates; P2P [120], a poison-to-poison remedy for backdoor defense; and SCOUT [1], which detects data-poisoning attempts during fine-tuning.

7.3 G-C: Inference-Time Guardrails

Inference-time guardrails protect AI systems during deployment by monitoring and controlling interactions between users, the LLM, and the external environment. Unlike pre-training and alignment defenses, these mechanisms operate at runtime to detect prompt injection, jailbreak attempts, malicious tool outputs, and unsafe model responses before they can influence the agent's reasoning or execution pipeline.

Protected Component. The core protected components are user prompts, system prompts, reasoning context, model responses and runtime interactions between the LLM and external environments.

Protected Trust Boundary. G-C guardrails protect the *Context Boundary* by making sure that untrusted runtime inputs do not influence a model's reasoning process or compromise its decision making during deployment.

Representative Defense Mechanisms. Inference-time defences approach the problem from multiple angles. LlamaGuard [46] performs prompt and response classification for safety. PromptShield [47] aims at direct and indirect prompt injection . Constitutional AI [4] instead limits behaviour through principle-based self-criticism. Framework-level tools like NeMo Guardrails [76] enforce policies on conversation and tool execution, while Guardrails AI [20] validates inputs and outputs against user-defined schemas. DataSentinel [64] monitors runtime inputs for adversarial patterns. Other works include design patterns for securing LLM agents against injection [6], Attention Tracker [45] that detects unusual attention patterns, SecurityLingua [60] that compresses the prompts to remove malicious content, and AdaptiveGuard [110] that adapts runtime safety policies. A recent study offers a more comprehensive assessment of the effectiveness of jailbreak guardrails [96] while another suggests a proactive defence against jailbreak [121]. These mechanisms work together to improve deployed LLMs against inference-time manipulation without affecting normal functionality. Recent work on reasoning-heavy models further demonstrates that long chain-of-thought generation introduces safety failure modes that standard input/output classifiers miss [49].

7.4 G-D: Memory and Knowledge Guardrails

Memory and knowledge guardrails preserve the integrity of external knowledge sources that augment the reasoning power of autonomous AI agents. Unlike the static parameters of the foundation model, these knowledge repositories are updated dynamically during deployment and may include retrieval-augmented generation (RAG) systems, vector databases, episodic memory, long-term

memory, knowledge graphs, and external document repositories. Therefore, these guardrails aim to ensure that retrieved information is reliable before it influences the agent's reasoning process.

Protected Component. The main protected components are vector databases, retrieval indices, external knowledge repositories, episodic memory, long-term memory, and other persistent knowledge stores used by autonomous AI agents.

Protected Trust Boundary. G-D guardrails protect the *Context Boundary* by preventing malicious manipulation of the retrieved knowledge and stored memory before they are integrated into the agent's reasoning process.

Representatives Defense Mechanisms. Representative approaches include TrustRAG [122] which validates retrieved documents via trust-aware retrieval, A-MemGuard [100] which detects memory poisoning by monitoring abnormal memory updates, MAGE [97] which evaluates the reliability of retrieved evidence before reasoning, RevPRAG [87] which improves retrieval robustness against poisoned knowledge sources through evidence verification, and activation-based memory integrity verification techniques which identify malicious knowledge before it is incorporated into the agent's internal reasoning state. Complementary work includes certifiably robust RAG against retrieval corruption [107], RAGuard [53] that presents a layered defence framework against RAG data poisoning, and work on tracing and attributing poisoned knowledge back to its source within RAG pipelines [115, 116]. Together, these mechanisms improve the robustness of retrieval and persistent memory while alleviating the threat of knowledge poisoning attacks in autonomous AI agents.

7.5 G-E: Multi-Agent Coordination Guardrails

Multi-agent coordination guardrails protect the communication and coordination mechanisms that enable multiple autonomous agents to cooperate toward a common objective. As AI pentesting frameworks evolve toward multi-agent architectures, agents communicate through shared channels, exchanging plans, observations, retrieved knowledge and execution results. These guardrails are aimed at preserving the integrity, authenticity and trustworthiness of inter-agent interactions and hence avoid compromised agents from affecting the collective decision-making process.

Protected Component. Important protected elements are communication channels between the agents, shared task representations, orchestration frameworks, coordination protocols, and trust relations between collaborating agents.

Protected Trust Boundary. G-E guardrails protect the *Execution Boundary* by verifying the authenticity, trustworthiness and resistance to manipulation of the messages exchanged between cooperating agents during the execution lifecycle.

Representative Defense Mechanisms. Representative approaches include cryptographic message authentication for verifying inter-agent communication, trust-aware agent verification frameworks that dynamically evaluate the reliability of participating agents, consensus-based coordination mechanisms that validate shared decisions before execution, and RouteGuard [108] that secures multi-agent communication paths by detecting malicious routing behaviours. Other techniques include verification protocols for communication, role-based authorisation mechanisms, and crossagent consistency checking that detect conflicting observations or abnormal patterns of coordination before they are propagated throughout the network of agents. Further approaches include AutoDefense [112], which coordinates multiple LLM agents to jointly filter jailbreak attempts; a multi-agent defense pipeline against prompt injection [38]; CoopGuard [58], which safeguards cooperative agents against evolving multi-round attacks; TrinityGuard [93], a unified framework for securing multi-agent systems; PSG-Agent [104], a personality-aware safety guardrail for LLM-based agents; and work on reconstructing cross-agent semantic flows for execution-aware

attack detection [101]. These mechanisms together increase the robustness of multi-agent systems against compromised agents and malicious message propagation with collaborative reasoning.

7.6 G-F: Tool and Execution Guardrails

Tool and execution guardrails protect the interfaces that autonomous AI agents use to interact with external environments. As opposed to typical LLM use cases where the model just generates text responses, AI pentesting agents run shell commands, call security tools, query external services, and coordinate with plugins via standardised interfaces like function calling and the Model Context Protocol (MCP). These guardrails confirm the generated actions before execution, reducing the risk that the actions are unsafe, unintended, or adversarially manipulated.

Protected Component. The primary protected elements include tool invocation interfaces, APIs for function-calling, modules for shell execution, plugin ecosystems, MCP servers, execution workflows, and integrations with external services.

Protected Trust Boundary. G-F guardrails secure the *Execution Boundary* by requiring validation of actions generated by LLMs before interfacing with external systems or security tools.

Representative Defense Mechanisms. Representative approaches include ToolSafe [68], which enforces proactive step-level guardrails and feedback before a tool call is allowed to execute; TraceSafe [11], which evaluates guardrail effectiveness across full multi-step tool-calling trajectories rather than individual calls in isolation; work on enforcing temporal constraints for LLM agents [52], which restricts the ordering and timing of tool invocations to prevent unsafe execution sequences; MindGuard [98], which inspects an agent's internal decision process to detect metadata poisoning before a tool is invoked; and SafeAgent [62], which provides a runtime protection layer that mediates and validates tool and plugin execution for agentic systems. These mechanisms, working together, reduce the likelihood of unauthorised tools being run, workflows being tampered with, the supply-chain being compromised, and malicious plugins being exploited, while retaining the necessary operational capabilities for autonomous penetration testing.

The qualitative assessment in Tables 4 and 5 is derived from the architectural design and guardrail mechanisms explicitly documented in each framework's publication, rather than empirical attack evaluation. Each entry identifies the dominant architectural factor limiting guardrail effectiveness in that category; "—" denotes structural inapplicability.

8 Research Gaps and Future Directions

As analysed in the sections above, existing guardrail mechanisms are disjoint across the layers of LLM lifecycle and agent architecture. Despite great progress in the mitigation of individual attack classes, current defences mostly operate in isolation and only partially protect autonomous AI pentesting agents. The results of the evaluation in Tables 4 and 5 suggest some promising avenues for future research.

RG1 — Integrated architecture-aware guardrail frameworks. Existing guardrails generally focus on protecting individual components like training pipelines, inference prompts, memory stores or external tool interfaces. However, autonomous AI pentesting agents integrate all these components within a single execution pipeline. Future research should develop unified guardrail frameworks capable of simultaneously securing the LLM lifecycle layer and the agent architecture layer while providing end-to-end protection across multiple trust boundaries.

RG2 — Runtime verification of foundation model integrity. Although numerous techniques have been proposed for detecting pre-training poisoning, alignment manipulation, and model backdoors, these approaches are largely designed for offline model evaluation. Little attention has been devoted to continuously verifying the integrity of foundation models deployed within autonomous

Table 4. Assessment of guardrail insufficiencies in representative single-agent AI pentesting frameworks. G-A–G-C: LLM Lifecycle Guardrails; G-D–G-F: Agent Architecture Guardrails.

Tool (Year)	Architecture	LLM Lifecycle			Agent Architecture			Research Gap
		G-A	G-B	G-C	G-D	G-E	G-F	
PentestGPT (2024) [18]	Parsing–Reasoning–Genera	FM	FM	OFN ENV	UNT	—	SEM	Human in the Loop reduces unsafe execution; memory and tool-call authorization remain weak.
AutoAttacker (2024) [109]	Planner + Retrieval-Augmented Generation (RAG)	FM	FM	OFN ENV	UNT	—	SEM	Unverified RAG memory and unrestricted tool invocation dominate.
ARACNE (2025) [70]	Planner + Interpreter + Tool Execution	FM	FM	OFN	—	—	SEM	Injection resistance improved; SSH execution lacks authorization control.
HackSynth (2024) [69]	Planner + Command Generation + Summarization	FM	FM	OFN ENV	—	—	SEM	Autonomous shell execution without verification is the primary concern.
PenHeal (2023) [40]	Pentest + Remediation	FM	FM	OFN	—	—	SEM	Partial validation cannot fully prevent adversarial influence on remediation.
HackingBuddyGPT (2023) [31]	SSH Executor + Feedback	FM	FM	OFN ENV	—	—	SEM	Direct shell execution without validation yields the highest exposure.
CHECKMATE (2025) [94]	Planner + Validator	FM	FM	OFN	—	—	SEM	Validation-guided reasoning improves robustness, but execution authorization remains limited.
Cochise (2026) [33]	Planner + Tool Execution + Benchmark Evaluation	FM	FM	OFN	—	—	SEM	Controlled evaluation reduces attack exposure, although tool authorization is not explicitly enforced.
RefPentester (2025) [15]	Planner + Reflection	FM	FM	OFN	—	—	SEM	Self-reflection improves planning reliability, but prompt injection and unsafe tool execution remain possible.

Legend: FM=Inherited protection from proprietary foundation models (guardrails are externally defined and cannot be independently verified or customized); **OFN**=Conflict between offensive penetration-testing objectives and foundation-model safety filtering; **ENV**=Adversarial observations accumulated during interaction that bias subsequent reasoning and planning; **UNT**=Untrusted memory or knowledge provenance without integrity verification; **SEM**=Semantic ambiguity in generated commands prior to execution, increasing the risk of unsafe tool invocation.

Note: G-A and G-B reflect protection inherited from the underlying foundation model rather than framework-level guardrails. G-E is structurally inapplicable to single-agent systems, as no inter-agent communication channel exists.

Color Coding: High Guardrail Insufficiency; Moderate Guardrail Insufficiency; Low Guardrail Insufficiency; Not Applicable

AI pentesting agents. Runtime verification techniques capable of detecting compromised model behaviour remain an important open challenge.

RG3 — Execution-aware safety reasoning. Current guardrail mechanisms predominantly analyze textual prompts and responses while providing limited reasoning about the operational semantics of autonomous tool execution. Future guardrails should incorporate execution-aware reasoning capable of evaluating command intent, execution context, authorization boundaries, and the potential security impact of downstream actions before tool invocation.

RG4 — Trustworthy memory and knowledge management. Persistent memory, retrieval-augmented generation (RAG), and shared knowledge repositories have become fundamental components of autonomous AI pentesting agents. However, memory provenance verification, integrity validation, continual consistency checking, and anomaly detection remain relatively immature. An important direction of research is to design trustworthy memory management architectures that are robust against long-term poisoning attacks.

RG5 — Secure multi-agent coordination. With the growing use of multi-agent pentesting frameworks, the demand for reliable cooperation between autonomous agents is crucial. Existing

Table 5. Assessment of guardrail insufficiencies in representative multi-agent AI pentesting frameworks. G-A–G-C: LLM Lifecycle Guardrails; G-D–G-F: Agent Architecture Guardrails.

Tool (Year)	Architecture	LLM Lifecycle			Agent Architecture			Research Gap
		G-A	G-B	G-C	G-D	G-E	G-F	
VulnBot (2025) [54]	Penetration Testing Graph (PTG) + Retrieval-Augmented Generation (RAG) + Multi-Agent	FM	FM	OFN	UNT	ROL	SEM	Unverified task-graph coordination and RAG memory enable cross-agent propagation.
BreachSeek (2024) [2]	Supervisor + Specialist Agents	FM	FM	OFN	UNT	ROL	SEM	Absence of trust verification enables adversarial propagation across specialists.
PentestAgent (2024) [79]	Reconnaissance Agent + Search Agents + Retrieval-Augmented Generation (RAG)	FM	FM	OFN	UNT	ROL	SEM	Untrusted retrieval and unrestricted messaging remain dominant weaknesses.
Incalmo (2025) [81]	Planner + Attack Graph + Specialist Agents	FM	FM	OFN	UNT	ROL	SEM	Modular isolation reduces propagation; cross-stage trust verification remains limited.
ReaperAI (2024) [89]	Autonomous Multi-Phase	FM	FM	OFN	UNT	ROL	SEM	Full autonomy lacks runtime verification for coordination and execution.
CAI (2025) [67]	Multi-Agent + Model Context Protocol (MCP)	FM	FM	OFN	UNT	ROL	SEM	Model diversity and unrestricted MCP/plugin execution yield the broadest attack surface.
AutoPentest (2024) [36]	Penetration Testing Tree (PTT) + Autonomous Multi-Agent	FM	FM	OFN	UNT	ROL	SEM	Autonomous agent collaboration lacks runtime trust verification and secure tool authorization.
PENTEST-AI (2025) [7]	MITRE ATT&CK-Guided Multi-Agent.	FM	FM	OFN	UNT	ROL	SEM	Collaborative reasoning lacks authenticated coordination and secure memory validation mechanisms.
AutoPentester (2025) [27]	Planner + Supervisor + Specialized Workers + RAG.	FM	FM	OFN	UNT	ROL	SEM	Supervisor-mediated task delegation and RAG retrieval lack authenticated coordination and provenance verification.
PTFusion (2025) [95]	MasterAgent + ReconAgents	FM	FM	OFN	UNT	ROL	SEM	Distributed reconnaissance increases shared-memory exposure while inter-agent trust verification remains incomplete.
xOffense (2025) [65]	Collaborative Multi-Agent	FM	FM	OFN	UNT	ROL	SEM	Extensive multi-agent collaboration and heterogeneous tool integration lack unified guardrail enforcement.

Legend: FM=Inherited protection from proprietary foundation models (guardrails are externally defined and cannot be independently verified or customized); **OFN**=Conflict between offensive penetration-testing objectives and foundation-model safety filtering; **ENV**=Adversarial observations accumulated during interaction that bias subsequent reasoning and planning; **UNT**=Untrusted memory or knowledge provenance without integrity verification; **ROL**=Absence of role-based trust verification between cooperating agents, allowing a compromised agent to influence others without authentication; **SEM**=Semantic ambiguity in generated commands prior to execution, increasing the risk of unsafe tool invocation.

Note: G-A and G-B reflect protection inherited from the underlying foundation model rather than framework-level guardrails. G-D–G-F assess framework-specific guardrails governing persistent memory, multi-agent coordination, and autonomous tool execution.

Color Coding: High Guardrail Insufficiency; Moderate Guardrail Insufficiency; Low Guardrail Insufficiency; Not Applicable

systems offer little or no support for agent authentication, message integrity validation, trust establishment or collaborative decision validation. Further research should focus on safe communication protocols, trust-aware coordination methods, and defences against adversarial message passing and worm-like attacks in collaborative AI ecosystems.

RG6 — Secure execution and invocation control of tools AI pentesting agents are increasingly using external plugins, Model Context Protocol (MCP) servers, browser automation, and autonomous shell execution, all of which significantly increase the attack surface. Existing guardrails generally do not attempt to verify execution intent or enforce fine-grained authorisation

policies prior to invoking external tools. We recommend the development of policy-aware execution frameworks that can validate tool requests and enforce operational constraints and prevent unauthorised or unintentional actions in future work.

RG7 — Standardised benchmarks and evaluation methodologies. While existing evaluation frameworks such as AgentDojo [16], AgentLAB [50], LITMUS [117] and work measuring agent progress on multi-step cyber attack scenarios [24] capture important aspects of autonomous agents, they do not offer a complete evaluation of AI pentesting systems in light of the taxonomy we propose. Future work should create standardised benchmarks, attack suites, and quantitative metrics to evaluate the effectiveness of guardrails, attack containment, execution safety, and long-term system robustness in realistic adversarial settings.

RG8 — Governance, auditability, and accountability. In addition to technical defences, the safe deployment of autonomous AI pentesting agents requires rigorous governance mechanisms. Standardised techniques for execution auditing, decision provenance, memory traceability, authorisation logging, and accountability have been little explored. The development of such governance frameworks will be critical in enabling safe and transparent deployment of autonomous offensive artificial intelligence systems in the real world.

Frameworks like RedTeamLLM [10] and more general treatments of offensive-security ideas and practices for artificial intelligence [35] show early efforts at operationalising agentic red-teaming, but do not yet include the architecture-aware guardrail coverage that this survey argues is necessary.

Overall, these research gaps indicate that the current guardrail mechanisms are still mostly component-specific and provide limited protection against cross-layer attacks on autonomous AI pentesting agents. Tackling these challenges will require integrated, architecture-aware defence frameworks that can simultaneously secure foundation models, persistent memory, multi-agent collaboration, and autonomous tool execution. Developing such holistic guardrail architectures is a key research direction toward trustworthy autonomous AI-based penetration testing systems.

9 Conclusion

The rapid evolution of large language models has revolutionised the field of AI-based penetration testing from interactive assistant systems to autonomous agents who can reason, plan, manage memory and execute tasks with the aid of tools. These capabilities significantly enhance the efficiency of penetration testing, but they also present a wide and dynamic attack surface beyond the traditional realm of LLM security. Therefore, the protection of autonomous AI pentesting agents has to take into account defences that span both the foundation model lifecycle and the agent architecture covering vulnerabilities.

This survey presents a broad analysis of the security landscape of autonomous AI pentesting agents. Firstly, we reviewed typical frameworks for single-agent and multi-agent systems, outlining their architectural designs and operational characteristics. Based on this analysis, we proposed a two-axis attack taxonomy that distinguishes vulnerabilities introduced by the *LLM Lifecycle Layer* (pre-training, fine-tuning, and inference-time attacks) from those introduced by the *Agent Architecture Layer* (memory and knowledge poisoning, multi-agent coordination attacks, and tool and execution attacks). This taxonomy offers a unified view for systematically classifying not only inherited vulnerabilities of LLMs but also threats specific to the agent.

We applied this taxonomy to assess representative AI pentesting frameworks and showed that the recent systems, despite their diverse architectural design, remain largely exposed to several attack categories. We further proposed a complementary guardrail taxonomy and studied how well the attack surfaces are mitigated by existing defensive mechanisms. Our analysis reveals that existing guardrails are still isolated and mainly protect isolated components, providing little protection against attacks that propagate through multiple trust boundaries.

The research gaps discussed in this survey also suggest that future AI pentesting agents will need integrated, architecture-aware guardrail frameworks that can simultaneously protect foundation models, persistent memory, multi-agent collaboration, and autonomous tool execution. Furthermore, standardised evaluation benchmarks, reliable governance mechanisms, and runtime verification techniques will be necessary for the reliable deployment of autonomous offensive artificial intelligence systems.

In summary, this survey offers a unified framework for understanding the evolving attack landscape facing AI-based penetration testing agents, and makes the case that securing them demands holistic mechanisms addressing both LLM lifecycle vulnerabilities and agent-specific architectural risks together. We hope the taxonomies, comparative analyses, and research directions presented here offer a useful foundation for future work toward building reliable, secure, and resilient autonomous AI pentesting systems.

References

[1] Mohamed Afane, Abhishek Satyam, Ke Chen, Tao Li, Junaid Farooq, and Juntao Chen. 2025. SCOUT: A Defense Against Data Poisoning Attacks in Fine-Tuned Language Models. *arXiv preprint arXiv:2512.10998* (2025).

[2] Ibrahim Alshehri, Adnan Alshehri, Abdulrahman Almalki, Majed Bamardouf, and Alaqsa Akbar. 2024. Breachseek: A multi-agent automated penetration tester. *arXiv preprint arXiv:2409.03789* (2024).

[3] Biagio Andreucci and Arcangelo Castiglione. 2026. Automation-Exploit: A Multi-Agent LLM Framework for Adaptive Offensive Security with Digital Twin-Based Risk-Mitigated Exploitation. *arXiv preprint arXiv:2604.22427* (2026).

[4] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional ai: Harmlessness from ai feedback, 2022. *URL https://arxiv. org/abs/2212.08073* 2212 (2022).

[5] Tim Baumgärtner, Yang Gao, Dana Alon, and Donald Metzler. 2024. Best-of-venom: Attacking rlhf by injecting poisoned preference data. *arXiv preprint arXiv:2404.05530* (2024).

[6] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, et al. 2025. Design patterns for securing llm agents against prompt injections. *arXiv preprint arXiv:2506.08837* (2025).

[7] Stanislas G Bianou and Rodrigue G Batogna. 2024. Pentest-ai, an llm-powered multi-agents framework for penetration testing automation leveraging mitre attack. In *2024 IEEE International Conference on Cyber Security and Resilience (CSR)*. IEEE, 763–770.

[8] Dillon Bowen, Brendan Murphy, Will Cai, David Khachaturov, Adam Gleave, and Kellin Pelrine. 2024. Scaling Laws for Data Poisoning in LLMs. *arXiv preprint arXiv:2408.02946* (2024).

[9] Nicholas Carlini, Matthew Jagielski, Christopher A. Choquette-Choo, Daniel Paleka, Will Pearce, Hyrum Anderson, Andreas Terzis, Kurt Thomas, and Florian Tramèr. 2024. Poisoning Web-Scale Training Datasets is Practical. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 407–425.

[10] Brian Challita and Pierre Parrend. 2025. RedTeamLLM: an Agentic AI framework for offensive security. In *IFIP International Workshop on Artificial Intelligence for Knowledge Management*. Springer, 337–354.

[11] Yen-Shan Chen, Sian-Yao Huang, Cheng-Lin Yang, and Yun-Nung Chen. 2026. Tracesafe: A systematic assessment of llm guardrails on multi-step tool-calling trajectories. *arXiv preprint arXiv:2604.07223* (2026).

[12] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. *Advances in Neural Information Processing Systems* 37 (2024), 130185–130213.

[13] Pengzhou Cheng, Yidong Ding, Tianjie Ju, Zongru Wu, Wei Du, Ping Yi, Zhuosheng Zhang, and Gongshen Liu. 2024. Trojanrag: Retrieval-augmented generation can be backdoor driver in large language models. *arXiv preprint arXiv:2405.13401* (2024).

[14] Kexin Chu. 2026. A Systematic Survey of Security Threats and Defenses in LLM-Based AI Agents: A Layered Attack Surface Framework. *arXiv e-prints* (2026), arXiv–2604.

[15] Hanzheng Dai, Yuanliang Li, Jun Yan, and Zhibo Zhang. 2025. Refpentester: A knowledge-informed self-reflective penetration testing framework based on large language models. In *2025 22nd Annual International Conference on Privacy, Security, and Trust (PST)*. IEEE, 1–8.

[16] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems* 37 (2024), 82895–82920.

[17] Gelei Deng, Yi Liu, Yuekang Li, Ruozhao Yang, Xiaofei Xie, Jie Zhang, Han Qiu, and Tianwei Zhang. 2026. What Makes a Good LLM Agent for Real-world Penetration Testing? *arXiv preprint arXiv:2602.17622* (2026).

[18] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security 24)*. 847–864.

[19] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2026. Memory injection attacks on LLM agents via query-only interaction. *Advances in Neural Information Processing Systems* 38 (2026), 46697–46731.

[20] Yi Dong, Ronghui Mu, Gaojie Jin, Yi Qi, Jinwei Hu, Xingyu Zhao, Jie Meng, Wenjie Ruan, and Xiaowei Huang. 2024. Building guardrails for large language models. *arXiv preprint arXiv:2402.01822* (2024).

[21] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. 2024. Llm agents can autonomously hack websites. *arXiv preprint arXiv:2402.06664* (2024).

[22] Neil Fendley, Edward W Staley, Joshua Carney, William Redman, Marie Chau, and Nathan Drenkow. 2025. A systematic review of poisoning attacks against large language models. *arXiv preprint arXiv:2506.06518* (2025).

[23] Mohamed Amine Ferrag, Abderrahmane Lakas, Norbert Tihanyi, and Merouane Debbah. 2026. Securing LLM Agents: From Prompt Sanitization to Autonomous Red Teaming and Beyond. *Internet of Things and Cyber-Physical Systems* (2026).

[24] Linus Folkerts, Will Payne, Simon Inman, Philippos Giavridis, Joe Skinner, Sam Deverett, James Aung, Ekin Zorer, Michael Schmatz, Mahmoud Ghanem, et al. 2026. Measuring AI Agents' Progress on Multi-Step Cyber Attack Scenarios. *arXiv preprint arXiv:2603.11214* (2026).

[25] Tingchen Fu, Mrinank Sharma, Philip Torr, Shay B Cohen, David Krueger, and Fazl Barez. 2024. Poisonbench: Assessing large language model vulnerability to data poisoning. *arXiv preprint arXiv:2410.08811* (2024).

[26] Pranav Gade, Simon Lermen, Charlie Rogers-Smith, and Jeffrey Ladish. 2023. Badllama: cheaply removing safety fine-tuning from llama 2-chat 13b. *arXiv preprint arXiv:2311.00117* (2023).

[27] Yasod Ginige, Akila Niroshan, Sajal Jain, and Suranga Seneviratne. 2025. Autopentester: An llm agent-based framework for automated pentesting. In *2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 163–174.

[28] Sahil Girhepuje, Aviral Verma, and Gaurav Raina. 2024. A survey on offensive ai within cybersecurity. *arXiv preprint arXiv:2410.03566* (2024).

[29] Xiangming Gu, Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Ye Wang, Jing Jiang, and Min Lin. 2024. Agent smith: A single image can jailbreak one million multimodal llm agents exponentially fast. *arXiv preprint arXiv:2402.08567* (2024).

[30] Thilo Hagendorff, Erik Derner, and Nuria Oliver. 2026. Large reasoning models are autonomous jailbreak agents. *Nature Communications* (2026).

[31] Andreas Happe and Jürgen Cito. 2023. Getting pwn'd by ai: Penetration testing with large language models. In *Proceedings of the 31st ACM joint european software engineering conference and symposium on the foundations of software engineering*. 2082–2086.

[32] Andreas Happe and Jürgen Cito. 2025. Benchmarking Practices in LLM-driven Offensive Security: Testbeds, Metrics, and Experiment Design. *arXiv preprint arXiv:2504.10112* (2025).

[33] Andreas Happe and Jürgen Cito. 2026. Cochise: A Reference Harness for Autonomous Penetration Testing. *arXiv preprint arXiv:2605.11671* (2026).

[34] Andreas Happe, Aaron Kaplan, and Juergen Cito. 2026. Llms as hackers: Autonomous linux privilege escalation attacks. *Empirical Software Engineering* 31, 3 (2026), 70.

[35] Josh Harguess and Chris M Ward. 2025. Offensive security for AI systems: concepts, practices, and applications. In *Assurance and Security for AI-enabled Systems 2025*, Vol. 13476. SPIE, 98–108.

[36] Julius Henke. 2025. Autopentest: Enhancing vulnerability management with autonomous llm agents. *arXiv preprint arXiv:2505.10321* (2025).

[37] Almog Hilel, Idan Shenfeld, Jacob Andreas, and Leshem Choshen. 2025. LLM Hypnosis: Exploiting User Feedback for Unauthorized Knowledge Injection to All Users. *arXiv preprint arXiv:2507.02850* (2025).

[38] SM Hossain, Ruksat Khan Shayoni, Mohd Ruhul Ameen, Akif Islam, MF Mridha, and Jungpil Shin. 2025. A multi-agent LLM defense pipeline against prompt injection attacks. *arXiv preprint arXiv:2509.14285* (2025).

[39] Chia-Yi Hsu, Yu-Lin Tsai, Chih-Hsun Lin, Pin-Yu Chen, Chia-Mu Yu, and Chun-Ying Huang. 2024. Safe lora: The silver lining of reducing safety risks when finetuning large language models. *Advances in Neural Information Processing Systems* 37 (2024), 65072–65094.

[40] Junjie Huang and Quanyan Zhu. 2023. Penheal: A two-stage llm framework for automated pentesting and optimal remediation. In *Proceedings of the workshop on autonomous cybersecurity*. 11–22.

[41] Tiansheng Huang, Gautam Bhattacharya, Pratik Joshi, Josh Kimball, and Ling Liu. 2024. Antidote: Post-fine-tuning safety alignment for large language models against harmful fine-tuning. *arXiv preprint arXiv:2408.09600* (2024).

[42] Tiansheng Huang, Sihao Hu, Fatih Ilhan, Selim Tekin, and Ling Liu. 2025. Booster: Tackling harmful fine-tuning for large language models via attenuating harmful perturbation. In *International Conference on Learning Representations*, Vol. 2025. 67202–67226.

[43] Tiansheng Huang, Sihao Hu, and Ling Liu. 2024. Vaccine: Perturbation-aware alignment for large language models against harmful fine-tuning attack. *Advances in Neural Information Processing Systems* 37 (2024), 74058–74088.

[44] Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M Ziegler, Tim Maxwell, Newton Cheng, et al. 2024. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. *arXiv preprint arXiv:2401.05566* (2024).

[45] Kuo-Han Hung, Ching-Yun Ko, Ambrish Rawat, I-Hsin Chung, Winston H Hsu, and Pin-Yu Chen. 2025. Attention tracker: Detecting prompt injection attacks in llms. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 2309–2322.

[46] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. 2023. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674* (2023).

[47] Dennis Jacob, Hend Alzahrani, Zhanhao Hu, Basel Alomair, and David Wagner. 2024. Promptshield: Deployable detection for prompt injection attacks. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*. 341–352.

[48] Sushil Jajodia, Steven Noel, Pramod Kalapa, Massimiliano Albanese, and John Williams. 2011. Cauldron mission-centric cyber situational awareness with defense in depth. In *2011-MILCOM 2011 military communications conference*. IEEE, 1339–1344.

[49] Fengqing Jiang, Zhangchen Xu, Yuetai Li, Luyao Niu, Zhen Xiang, Bo Li, Bill Yuchen Lin, and Radha Poovendran. 2025. Safechain: Safety of language models with long chain-of-thought reasoning capabilities. In *Findings of the Association for Computational Linguistics: ACL 2025*. 23303–23320.

[50] Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. 2026. Agentlab: Benchmarking llm agents against long-horizon attacks. *arXiv preprint arXiv:2602.16901* (2026).

[51] Hao Jing, Fanxiao Li, Yunyun Dong, Wei Zhou, and Renyang Liu. 2026. Memory poisoning attacks on retrieval-augmented Large Language Model agents via deceptive semantic reasoning. *Engineering Applications of Artificial Intelligence* 167 (2026), 113968.

[52] Adharsh Kamath, Sishen Zhang, Calvin Xu, Shubham Ugare, Gagandeep Singh, and Sasa Misailovic. 2025. Enforcing temporal constraints for llm agents. *arXiv preprint arXiv:2512.23738* (2025).

[53] Tanish Kolhe, Pushkal Kumar, Tucker Nielson, Shubham Zala, Vincent Li, Michael Saxon, Sean Wu, and Kevin Zhu. 2025. RAGuard: A Layered Defense Framework for Retrieval-Augmented Generation Systems Against Data Poisoning. In *Socially Responsible and Trustworthy Foundation Models at NeurIPS 2025*.

[54] He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. 2025. Vulnbot: Autonomous penetration testing for a multi-agent collaborative framework. *arXiv preprint arXiv:2501.13411* (2025).

[55] Hongyi Li, Jiawei Ye, Jie Wu, Tianjie Yan, Chu Wang, and Zhixin Li. 2025. JailPO: A Novel Black-box Jailbreak Framework via Preference Optimization against Aligned LLMs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 27419–27427.

[56] Jing-Jing Li, Jianfeng He, Chao Shang, Devang Kulshreshtha, Xun Xian, Yi Zhang, Hang Su, Sandesh Swamy, and Yanjun Qi. 2025. STAC: When Innocent Tools Form Dangerous Chains to Jailbreak LLM Agents. *arXiv preprint arXiv:2509.25624* (2025).

[57] Mingjie Li, Wai Man Si, Michael Backes, Yang Zhang, and Yisen Wang. 2025. Salora: Safety-alignment preserved low-rank adaptation. *arXiv preprint arXiv:2501.01765* (2025).

[58] Siyuan Li, Zehao Liu, Xi Lin, Qinghua Mao, Yuliang Chen, Haoyu Li, Jun Wu, Jianhua Li, and Xiu Su. 2026. CoopGuard: Stateful Cooperative Agents Safeguarding LLMs Against Evolving Multi-Round Attacks. *arXiv preprint arXiv:2604.04060* (2026).

[59] Xi Li, Ruofan Mao, Yusen Zhang, Renze Lou, Chen Wu, and Jiaqi Wang. 2025. Chain-of-scrutiny: Detecting backdoor attacks for large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*. 7705–7727.

[60] Yucheng Li, Surin Ahn, Huiqiang Jiang, Amir H Abdi, Yuqing Yang, and Lili Qiu. 2025. Securitylingua: Efficient defense of llm jailbreak attacks via security-aware prompt compression. *arXiv preprint arXiv:2506.12707* (2025).

[61] Yige Li, Hanxun Huang, Yunhan Zhao, Xingjun Ma, and Jun Sun. 2026. Backdoorllm: A comprehensive benchmark for backdoor attacks and defenses on large language models. *Advances in neural information processing systems* 38 (2026).

[62] Hailin Liu, Eugene Ilyushin, Jie Ni, and Min Zhu. 2026. SafeAgent: A runtime protection architecture for agentic systems. *arXiv preprint arXiv:2604.17562* (2026).

[63] Jingju Liu, Yue Zhang, Shicheng Zhou, Jiahai Yang, Yuliang Lu, and Xiaofeng Zhong. 2025. Autonomous penetration testing using reinforcement learning: A review and perspectives. *Expert Systems with Applications* (2025), 130219.

[64] Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. 2025. Datasentinel: A game-theoretic detection of prompt injection attacks. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2190–2208.

[65] Phung Duc Luong, Tran Gia Le Bao, Nguyen Vu Khai Tam, Dong Huu Nguyen Khoa, Nguyen Huu Quyen, Van-Hau Pham, et al. 2025. xOffense: An AI-driven autonomous penetration testing framework with offensive knowledge-enhanced LLMs and multi agent systems. *arXiv e-prints* (2025), arXiv–2509.

[66] Matteo Lupinacci, Francesco Aurelio Pironti, Francesco Blefari, Francesco Romeo, Luigi Arena, and Angelo Furfaro. 2025. The dark side of llms: Agent-based attacks for complete computer takeover. *arXiv preprint arXiv:2507.06850* (2025).

[67] Víctor Mayoral-Vilches, Luis Javier Navarrete-Lozano, María Sanz-Gómez, Lidia Salas Espejo, Martiño Crespo-Álvarez, Francisco Oca-Gonzalez, Francesco Balassone, Alfonso Glera-Picón, Unai Ayucar-Carbajo, Jon Ander Ruiz-Alcalde, et al. 2025. Cai: An open, bug bounty-ready cybersecurity ai. *arXiv preprint arXiv:2504.06017* (2025).

[68] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. 2026. ToolSafe: Enhancing Tool Invocation Safety of LLM-based agents via Proactive Step-level Guardrail and Feedback. *arXiv preprint arXiv:2601.10156* (2026).

[69] Lajos Muzsai, David Imolai, and András Lukács. 2024. Hacksynth: Llm agent and evaluation framework for autonomous penetration testing. *arXiv preprint arXiv:2412.01778* (2024).

[70] Tomas Nieponice, Veronica Valeros, and Sebastian Garcia. 2025. Aracne: An llm-based autonomous shell pentesting agent. *arXiv preprint arXiv:2502.18528* (2025).

[71] Chetan Pathade. 2025. Red teaming the mind of the machine: A systematic evaluation of prompt injection and jailbreak vulnerabilities in llms. *arXiv preprint arXiv:2505.04806* (2025).

[72] Jiaren Peng, Zeqin Li, Chang You, Yan Wang, Hanlin Sun, Xuan Tian, Shuqiao Zhang, Junyi Liu, Jianguo Zhao, Renyang Liu, et al. 2026. Hackers or Hallucinators? A Comprehensive Analysis of LLM-Based Automated Penetration Testing. *arXiv preprint arXiv:2604.05719* (2026).

[73] Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527* (2022).

[74] Senmao Qi, Yifei Zou, Peng Li, Ziyi Lin, Xiuzhen Cheng, and Dongxiao Yu. 2025. Amplified vulnerabilities: Structured jailbreak attacks on llm-based multi-agent debate. *arXiv preprint arXiv:2504.16489* (2025).

[75] Yubin Qu, Yi Liu, Tongcheng Geng, Gelei Deng, Yuekang Li, Leo Yu Zhang, Ying Zhang, and Lei Ma. 2026. Supply-Chain Poisoning Attacks Against LLM Coding Agent Skill Ecosystems. *arXiv preprint arXiv:2604.03081* (2026).

[76] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails. In *Proceedings of the 2023 conference on empirical methods in natural language processing: system demonstrations*. 431–445.

[77] Domenic Rosati, Jan Wehner, Kai Williams, Łukasz Bartoszcze, David Atanasov, Robie Gonzales, Subhabrata Majumdar, Carsten Maple, Hassan Sajjad, and Frank Rudzicz. 2024. Representation noising: A defence mechanism against harmful finetuning. *Advances in Neural Information Processing Systems* 37 (2024), 12636–12676.

[78] Han Shen, Pin-Yu Chen, Payel Das, and Tianyi Chen. 2025. Seal: Safety-enhanced aligned llm fine-tuning via bilevel data selection. In *International Conference on Learning Representations*, Vol. 2025. 31243–31264.

[79] Xiangmin Shen, Lingzhi Wang, Zhenyuan Li, Yan Chen, Wencheng Zhao, Dawei Sun, Jiashui Wang, and Wei Ruan. 2025. Pentestagent: Incorporating llm agents to automated penetration testing. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. 375–391.

[80] Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. 2025. Prompt injection attack to tool selection in llm agents. *arXiv preprint arXiv:2504.19793* (2025).

[81] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. 2025. Incalmo: An Autonomous LLM-assisted System for Red Teaming Multi-Host Networks. *arXiv preprint arXiv:2501.16466* (2025).

[82] Alexandra Souly, Javier Rando, Ed Chapman, Xander Davies, Burak Hasircioglu, Ezzeldin Shereen, Carlos Mougan, Vasilios Mavroudis, Erik Jones, Chris Hicks, et al. 2025. Poisoning attacks on LLMs require a near-constant number of poison samples. *arXiv preprint arXiv:2510.07192* (2025).

[83] Devansh Srivastav and Xiao Zhang. 2025. Safe in isolation, dangerous together: Agent-driven multi-turn decomposition jailbreaks on llms. In *Proceedings of the 1st Workshop for Research on Agent Language Models (REALM 2025)*. 170–183.

[84] Zhen Sun, Tianshuo Cong, Yule Liu, Chenhao Lin, Xinlei He, Rongmao Chen, Xingshuo Han, and Xinyi Huang. 2025. Peftguard: Detecting backdoor attacks against parameter-efficient fine-tuning. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1713–1731.

[85] Balachandra Devarangadi Sunil, Isheeta Sinha, Piyush Maheshwari, Shantanu Todmal, Shreyan Mallik, and Shuchi Mishra. 2026. Memory poisoning attack and defense on memory based llm-agents. *arXiv preprint arXiv:2601.05504*

(2026).

[86] Rishub Tamirisa, Bhrugu Bharathi, Long Phan, Andy Zhou, Alice Gatti, Tarun Suresh, Maxwell Lin, Justin Wang, Rowan Wang, Ron Arel, et al. 2025. Tamper-resistant safeguards for open-weight llms. In *International Conference on Learning Representations*, Vol. 2025. 101802–101829.

[87] Xue Tan, Hao Luan, Mingyu Luo, Xiaoyan Sun, Ping Chen, and Jun Dai. 2024. Revprag: Revealing poisoning attacks in retrieval-augmented generation through llm activation analysis. *arXiv preprint arXiv:2411.18948* (2024).

[88] Khang Tran, Yazan Boshmaf, Issa Khalil, Hai Phan, Ting Yu, and Md Rizwan Parvez. 2026. Poisoning LLM-based Code Agents with Styles. *arXiv preprint arXiv:2605.27631* (2026).

[89] Leroy Jacob Valencia. 2024. Artificial intelligence as the new hacker: Developing agents for offensive security. *arXiv preprint arXiv:2406.07561* (2024).

[90] Mathew J Walter, Aaron Barrett, and Kimberly Tam. 2024. A red teaming framework for securing AI in maritime autonomous systems. *Applied Artificial Intelligence* 38, 1 (2024), 2395750.

[91] Alexander Wan, Eric Wallace, Sheng Shen, and Dan Klein. 2023. Poisoning language models during instruction tuning. In *International Conference on Machine Learning*. PMLR, 35413–35425.

[92] Jiongxiao Wang, Junlin Wu, Muhao Chen, Yevgeniy Vorobeychik, and Chaowei Xiao. 2024. Rlhfpoison: Reward poisoning attack for reinforcement learning with human feedback in large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2551–2570.

[93] Kai Wang, Biaojie Zeng, Zeming Wei, Chang Jin, Hefeng Zhou, Xiangtian Li, Chao Yang, Jingjing Qu, Xingcheng Xu, and Xia Hu. 2026. TrinityGuard: A unified framework for safeguarding multi-agent systems. *arXiv preprint arXiv:2603.15408* (2026).

[94] Lingzhi Wang, Xinyi Shi, Ziyu Li, Yi Jiang, Shiyu Tan, Yuhao Jiang, Junjie Cheng, Wenyuan Chen, Xiangmin Shen, Zhenyuan LI, et al. 2025. Automated Penetration Testing with LLM Agents and Classical Planning. *arXiv preprint arXiv:2512.11143* (2025).

[95] Wenhao Wang, Hao Gu, Zhixuan Wu, Hao Chen, Xingguo Chen, and Fan Shi. 2025. PTFusion: LLM-driven Context-Aware Knowledge Fusion for Web Penetration Testing. *Information Fusion* (2025), 103731.

[96] Xunguang Wang, Zhenlan Ji, Wenxuan Wang, Zongjie Li, Daoyuan Wu, and Shuai Wang. 2025. Sok: Evaluating jailbreak guardrails for large language models. *arXiv preprint arXiv:2506.10597* (2025).

[97] Yuhui Wang, Tanqiu Jiang, Jiacheng Liang, Charles Fleming, and Ting Wang. 2026. MAGE: Safeguarding LLM Agents against Long-Horizon Threats via Shadow Memory. *arXiv preprint arXiv:2605.03228* (2026).

[98] Z Wang, H Du, G Shi, J Zhang, H Cheng, Y Yao, K Guo, and XY Li. 2025. MindGuard: Intrinsic decision inspection for securing LLM agents against metadata poisoning. *arXiv preprint arXiv:2508.20412* (2025).

[99] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2026. Mcptox: A benchmark for tool poisoning on real-world mcp servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 35811–35819.

[100] Qianshan Wei, Tengchao Yang, Yaochen Wang, Xinfeng Li, Lijun Li, Zhenfei Yin, Yi Zhan, Thorsten Holz, Zhiqiang Lin, and XiaoFeng Wang. 2025. A-memguard: A proactive defense framework for llm-based agent memory. *arXiv preprint arXiv:2510.02373* (2025).

[101] Yangyang Wei, Yijie Xu, Zhenyuan Li, Xiangmin Shen, and Shouling Ji. 2026. Beyond Input Guardrails: Reconstructing Cross-Agent Semantic Flows for Execution-Aware Attack Detection. *arXiv preprint arXiv:2603.04469* (2026).

[102] Junlin Wu, Jiongxiao Wang, Chaowei Xiao, Chenguang Wang, Ning Zhang, and Yevgeniy Vorobeychik. 2025. Preference poisoning attacks on reward model learning. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1622–1640.

[103] Jiangrong Wu, Zitong Yao, Yuhong Nan, and Zibin Zheng. 2026. ChainFuzzer: Greybox Fuzzing for Workflow-Level Multi-Tool Vulnerabilities in LLM Agents. *arXiv preprint arXiv:2603.12614* (2026).

[104] Yaozu Wu, Jizhou Guo, Dongyuan Li, Henry Peng Zou, Wei-Chieh Huang, Yankai Chen, Zhen Wang, Weizhi Zhang, Yangning Li, Meng Zhang, et al. 2025. Psg-agent: Personality-aware safety guardrail for llm-based agents. *arXiv preprint arXiv:2509.23614* (2025).

[105] Zongru Wu, Pengzhou Cheng, Lingyong Fang, Zhuosheng Zhang, and Gongshen Liu. 2025. Gracefully filtering backdoor samples for generative large language models without retraining. In *Proceedings of the 31st International Conference on Computational Linguistics*. 3267–3282.

[106] Zongru Wu, Zhuosheng Zhang, Pengzhou Cheng, and Gongshen Liu. 2024. Acquiring clean language models from backdoor poisoned datasets by downscaling frequency space. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8116–8134.

[107] Chong Xiang, Tong Wu, Zexuan Zhong, David Wagner, Danqi Chen, and Prateek Mittal. 2024. Certifiably robust rag against retrieval corruption. *arXiv preprint arXiv:2405.15556* (2024).

[108] Wenjie Xiao, Xuehai Tang, Biyu Zhou, Songlin Hu, and Jizhong Han. 2026. RouteGuard: Internal-signal detection of skill poisoning in LLM agents. *arXiv preprint arXiv:2604.22888* (2026).

[109] Jiacen Xu, Jack W Stokes, Geoff McDonald, Xuesong Bai, David Marshall, Siyue Wang, Adith Swaminathan, and Zhou Li. 2024. Autoattacker: A large language model guided system to implement automatic cyber-attacks. *arXiv preprint arXiv:2403.01038* (2024).

[110] Rui Yang, Michael Fu, Chakkrit Tantithamthavorn, Chetan Arora, Gunel Gulmammadova, and Joey Chua. 2025. AdaptiveGuard: Towards Adaptive Runtime Safety for LLM-Powered Software. *arXiv preprint arXiv:2509.16861* (2025).

[111] Weichen Yu, Kai Hu, Tianyu Pang, Chao Du, Min Lin, and Matt Fredrikson. 2025. LLM-based Multi-Agents System Attack via Continuous Optimization with Discrete Efficient Search. In *Second Conference on Language Modeling*.

[112] Yifan Zeng, Yiran Wu, Xiao Zhang, Huazheng Wang, and Qingyun Wu. 2024. Autodefense: Multi-agent llm defense against jailbreak attacks. *arXiv preprint arXiv:2403.04783* (2024).

[113] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506.

[114] Boyang Zhang, Yicong Tan, Yun Shen, Ahmed Salem, Michael Backes, Savvas Zannettou, and Yang Zhang. 2025. Breaking agents: Compromising autonomous llm agents through malfunction amplification. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 34952–34964.

[115] Baolei Zhang, Haoran Xin, Yuxi Chen, Zhuqing Liu, Biao Yi, Tong Li, Lihai Nie, Zheli Liu, and Minghong Fang. 2025. Who taught the lie? responsibility attribution for poisoned knowledge in retrieval-augmented generation. *arXiv preprint arXiv:2509.13772* (2025).

[116] Baolei Zhang, Haoran Xin, Minghong Fang, Zhuqing Liu, Biao Yi, Tong Li, and Zheli Liu. 2025. Traceback of poisoning attacks to retrieval-augmented generation. In *Proceedings of the ACM on Web Conference 2025*. 2085–2097.

[117] Chiyu Zhang, Huiqin Yang, Bendong Jiang, Xiaolei Zhang, Yiran Zhao, Ruyi Chen, Lu Zhou, Xiaogang Xu, Jiafei Wu, Liming Fang, et al. 2026. LITMUS: Benchmarking Behavioral Jailbreaks of LLM Agents in Real OS Environments. *arXiv preprint arXiv:2605.10779* (2026).

[118] Wei Zhang, ZhenYuan Wei, ChenWei Liang, ShengNing Lang, Hao Yan, Zhimo Han, Yichao Zhang, Chen Yang, Zhenyu Yu, and Mujiangshan Wang. 2025. Learning-Based Automated Adversarial Red-Teaming for Robustness Evaluation of Large Language Models. *Available at SSRN 5965054* (2025).

[119] Yiming Zhang, Javier Rando, Ivan Evtimov, Jianfeng Chi, Eric Michael Smith, Nicholas Carlini, Florian Tramèr, and Daphne Ippolito. 2025. Persistent Pre-Training Poisoning of LLMs. In *The Thirteenth International Conference on Learning Representations*.

[120] Shuai Zhao, Xinyi Wu, Shiqian Zhao, Xiaobao Wu, Zhongliang Guo, Yanhao Jia, and Anh Tuan Luu. 2025. P2P: A Poison-to-Poison Remedy for Reliable Backdoor Defense in LLMs. *arXiv preprint arXiv:2510.04503* (2025).

[121] Weiliang Zhao, Jinjun Peng, Daniel Ben-Levi, Zhou Yu, and Junfeng Yang. 2025. Proactive defense against LLM Jailbreak. *arXiv preprint arXiv:2510.05052* (2025).

[122] Huichi Zhou, Kin-Hei Lee, Zhonghao Zhan, Yue Chen, Zhenhao Li, Zhaoyang Wang, Hamed Haddadi, and Emine Yilmaz. 2025. TrustRAG: enhancing robustness and trustworthiness in retrieval-augmented generation. *arXiv preprint arXiv:2501.00879* (2025).

[123] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. 2026. Teams of llm agents can exploit zero-day vulnerabilities. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. 23–35.

[124] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).

[125] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2025. Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models, 2024. *URL https://arxiv. org/abs/2402.07867* (2025).